

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-ad8f8893af168f1d4.jsonl`  
- SHA-256: `c9845b5fcec2f96ff1462df342fe33ae307d0b50a37f7b0e6a871ea5687a1b2`  
- Source modified: `2026-06-16T02:36:05+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:31:57.621Z)

You are the lead architect. Using the codebase understanding and the three design proposals below, produce ONE unified, coherent design plan for committed VIDEO-FREE golden-set regression. It MUST build on the existing infrastructure (cite the real modules from the grounding), pick concrete seam(s) and artifact formats, define a fixture layout + an extraction tool + auto-discovered parametrized regression test(s), specify BOTH a characterization/snapshot gate (refactor side-effect detection) and a quality-floor gate (vs golden labels, tied to ablation), define schema + label versioning & migration, a repo-size strategy, licensing/privacy handling, CI integration, the contributor-on-another-machine workflow, a growth story (how it scales as goldens are added), and a phased rollout where each phase is a small PR. Graft the best ideas from all three proposals and resolve conflicts explicitly. Recommend where this should be documented (extend a specific existing doc from the grounding, or a new doc ? name it). List open questions needing an OWNER decision (this repo is owner-gated on product/strategy/licensing). Be concrete and explicitly honest about limits.

UNDERSTANDING (JSON):

```
[  
  {  
    "area": "Existing regression/golden/eval infrastructure in backend/eval (badminton rally detector) ? for a \"video-free regression from committed structured files\" design",  
    "summary": "The repo already has a mature, layered, mostly-pure eval stack under backend/eval/ that is video-free at the SCORING boundary: pure interval metrics (metrics.py, segmentation_metrics.py), one canonical GT CSV loader (gt_loader.py), a DB-read harness (harness.py), a baseline-snapshot regression gate (regression.py + run_regression.py, exit 0/1/3) and a sibling eval harness CLI (run_eval.py), an A/B experiment engine (experiment.py) with a committed leaderboard, an ablation driver (ablation.py), and two offline replay-substrate producers (fusion_golden.py -> per-video *_golden_features.json; export_wasb_segments.py). HOWEVER, none of the existing flows runs from committed-in-repo structured files end-to-end. Two distinct \"sources of truth\" exist: (a) the DB path (harness/regression read predictions from a SQLite DB that only the GPU pipeline can populate), and (b) the trajectory/feature path (calibrate_wasb, distill_local, fusion_golden, served_gate_a, ablation read WASB trajectory CSVs / golden_features JSON / .rallies.csv whose paths come from output/human_lovo_manifest.json ? which is gitignored and points at absolute paths OUTSIDE the repo). ablation.py is the closest existing thing to a video-free, file-driven regression: it loads *_golden_features.json from a directory and runs entirely on CPU with no DB and no video ? but those JSONs are not committed. The committed structured artifacts that DO survive a clean checkout live in eval_baselines/ (tracked): heuristic_lovo_n6.json (the LOVO floor), experiment_leaderboard.json, gemini_wrapper_2026-06-10.json. A design can reuse the entire scoring/loader/stats/gate stack as-is and only needs to ADD committed golden fixtures (labels + a frozen predictions/feature substrate) plus a thin file-fed runner.",  
    "keyFindings": [  
      "(1) WHAT EXISTS / HOW RUN: Two CLI entrypoints at repo root. run_eval.py (-> backend.eval.harness.evaluate) scores DB-stored predictions vs a GT CSV and prints/JSON-dumps P/R/F1 per stage; read-only, no gate. run_regression.py (-> backend.eval.regression.regress_with_provider) scores DB predictions vs GT from an annotation
```

provider tier and compares to a snapshotted baseline JSON, returning exit 0=ok / 1=regressed / 3=no-baseline-to-compare. Both read predictions from SQLite (harness.get_predictions: 'candidates' -> db.query_candidate_segments, 'final' -> db.query_play_segments). backend.eval.experiment.compare/evaluate_variant is the A/B engine; backend.eval.ablation.run_ablation is the LOSO/LOGO driver (CLI: python -m backend.eval.ablation --features-dir output). fusion_golden, export_wasb_segments, served_gate_a, calibrate_wasb, distill_local are all 'python -m backend.eval.<mod>' CLIs. CI (.github/workflows/ci.yml) runs ONLY ruff + mypy + import-smoke + the full pytest suite (--cov-fail-under=70); it does NOT invoke run_regression/run_eval as a gate. So regression today is pytest-only (test_eval_regression.py) using inline tmp_path CSVs + in-memory Database, NEVER a committed golden run.",

"(2) ALREADY VIDEO-FREE? Partially, and in two senses. The SCORING math (metrics.py, segmentation_metrics.py) and the GT loader (gt_loader.py) are pure stdlib, no video, no GPU, no numpy. harness/regression are video-free in that they read predictions from the DB (no re-detect) ? but the DB itself can only be populated by running the GPU pipeline on video, so 'no committed file -> no run'. The CLOSEST existing video-free-from-files flow is ablation.py: load_videos(features_dir) reads every *_golden_features.json (intervals + shuttle/person/audio feature columns + golden gts baked in) and runs the full ablation 100% on CPU with NO DB and NO video. experiment.compare likewise runs purely on VideoCandidates objects (intervals+features+gts). So the ENGINE to do 'video-free regression from structured files' already exists end-to-end ? what is missing is COMMITTED inputs: the *_golden_features.json files are written to gitignored output/ and are not in the repo.",

"(3) GOLDEN FIXTURES: eval_baselines/ IS tracked (committed) ? NOT gitignored. It holds heuristic_lovo_n6.json (mean_f1 0.480 + per-video F1 floor for n=6 corpus), experiment_leaderboard.json (append-only A/B records with honest ?F1 CI + sign test), gemini_wrapper_2026-06-10.json. There is NO committed regression_baseline.json (regression.DEFAULT_BASELINE_PATH='eval_baselines/regression_baseline.json' but the file does not yet exist ? it would be committed there once created via --update-baseline). There are NO committed golden label CSVs (.rallies.csv), NO committed WASB trajectory CSVs, and NO committed *_golden_features.json. The golden corpus lives entirely OUTSIDE the repo: output/human_lovo_manifest.json (gitignored) lists 6 videos by name with absolute Windows paths like C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/*.csv and ../Golden Labelled/*.rallies.csv. Tests that need the golden corpus (test_served_gate_a_regression.py) skipif the manifest/files are absent (CI skips; owner box runs).",

"(4) ARTIFACT FORMATS ARE ALREADY STANDARDISED for a file-fed design: gt_loader.load_gt_intervals accepts BOTH golden CSV conventions through one reader (header start/end OR start_time/end_time, else positional col 0,1; MM:SS/HH:MM:SS or decimal seconds; strict raises with file:line, lenient logs skips). export_wasb_segments writes the golden/slicer schema CSV (rally_number, start_time, end_time, duration, start_mmss, end_mmss) plus a *_vs_golden comparison CSV (type, ref, start_time, end_time, mmss, status, iou, pred_start, pred_end, start_err_s, end_err_s). fusion_golden writes one JSON per video {name, fps, frame_width, candidates: [{start, end, shuttle{5 cols}, person{5 cols}, label}], gts: [[s, e]...]} consumed by ablation.load_videos and re-scorable forever with no GPU. The regression baseline JSON schema is {'<video_id>::<stage>': {video_id, stage, precision, recall, f1, iou_threshold, detector, gt_hash}} with incommensurability guards (IoU/detector/gt_hash mismatch -> flagged, not silently compared).",

"(5) FOR A 'VIDEO-FREE REGRESSION FROM COMMITTED STRUCTURED FILES' DESIGN ? REUSE vs ADD. REUSE (do not reinvent): metrics.score/match_intervals/pr_curve; segmentation_metrics.f1_at_overlaps/mean_average_precision/segment_count_ratio; gt_loader.load_gt_intervals (the one CSV reader); regression.regress/save_baseline/load_baselines/gt_hash + the exit 0/1/3 gate contract in run_regression.py; experiment.VideoCandidates + evaluate_variant + compare + record_experiment + the eval_baselines/ committed-location convention; ablation.load_videos (already reads committed-shaped golden_features JSON, DB-free, GPU-free); eval_stats CI+sign-test, splits.assert_disjoint_train/eval guard. ADD: (a) commit a small golden fixture set in-repo (label CSVs in the golden schema + a FROZEN predictions/feature substrate ? either tiny *_golden_features.json or a committed predictions JSON/CSV per video) under a tracked dir (tests/ has no fixtures today; eval_baselines/ or a new tests/golden/ are the natural homes); (b) a thin file-fed predictions loader that feeds metrics/regression WITHOUT the DB (today regression.regress only takes predictions via harness.get_predictions -> DB; you need a variant that reads predictions from a committed file); (c) commit the manifest with repo-relative paths (current output/human_lovo_manifest.json is gitignored + absolute-pathed); (d) optionally wire run_regression into CI (today it is not a CI gate)."

],

"dataStructures": [

```

{
    "name": "RegressionResult / regression_baseline.json",
    "where": "backend/eval/regression.py
(DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json ? file NOT yet committed)",
    "shapeOrFormat": "Baseline store JSON: { '<video_id>::<stage>':
{video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash} }. RegressionResult
dataclass adds regressed:bool, reasons:[str], has_baseline:bool, tolerance,
baseline_precision/recall. gt_hash = sha1 of sorted rounded GT intervals (detects label drift).
Incommensurability guards flag IoU/detector/gt_hash mismatch."
},
{
    "name": "Golden / slicer CSV schema",
    "where": "backend/eval/export_wasb_segments.py (SEG_FIELDS); read by
gt_loader.load_gt_intervals and tools.rally_slicer.load_rally_labels",
    "shapeOrFormat": "CSV columns:
rally_number,start_time,end_time,duration,start_mmss,end_mmss. The canonical GT reader
(gt_loader) also accepts the collector export form (start,end), positional headerless, and
MM:SS/HH:MM:SS or decimal-seconds timestamps. Comparison CSV (COMP_FIELDS):
type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s."
},
{
    "name": "*_golden_features.json (replay substrate)",
    "where": "produced by backend/eval/fusion_golden.py (run -> out_dir, default gitignored
output/); consumed by backend/eval/ablation.py load_videos and the experiment engine",
    "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end,
shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_co
urt,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
audio?:{audio_hit_count,audio_hit_rate,audio_hit_strength_mean}, label:int]],
gts:[{start,end,...}]. This is the GPU-free, video-free re-scoring input ? exactly the artifact
a file-driven regression would commit."
},
{
    "name": "human_lovo_manifest.json",
    "where": "output/human_lovo_manifest.json ? GITIGNORED (output/ is in .gitignore line
39); read by fusion_golden, served_gate_a, distill_local, calibrate_wasb CLIs",
    "shapeOrFormat": "JSON array of [{name, trajectory:<abs path to WASB CSV>, fps,
frame_width, labels:<abs path to .rallies.csv>}]. Paths are absolute and outside the repo
(C:/Users/avidu/Projects/Annotation Setup/...), so this corpus does NOT survive a clean
checkout. n=6 owned golden videos."
},
{
    "name": "VideoCandidates / Variant / experiment record",
    "where": "backend/eval/experiment.py; leaderboard committed at
eval_baselines/experiment_leaderboard.json",
    "shapeOrFormat": "VideoCandidates(name, intervals:[(s,e)], features:{col->[per-candidate
floats]}, gts:[(s,e)] or None). Variant = declarative re-scoring recipe
(min_crossings/min_players gates, feature_names, classifier_model, threshold, merge_gap; control
defaults are no-op). Leaderboard row: {timestamp,iou,label_set_hash,num_videos,variant_a/b:{name
,spec_hash,aggregate},delta,honest_delta_f1:{mean_delta,ci_low,ci_high,sign_test:{wins,losses,p
value}}}}."
},
{
    "name": "WASB trajectory CSV",
    "where": "parsed by TrackNetRunner.parse_trajectory_csv; consumed by
calibrate_wasb.make_predict_fn, export_wasb_segments, served_gate_a,
distill_local.build_candidates",
    "shapeOrFormat": "CSV columns: Frame,Visibility,X,Y (one row per frame). The per-video
raw substrate from which candidate windows + shuttle features are derived; lives outside the
repo per the gitignored manifest."
},
{
    "name": "split manifest (train/eval guard)",
    "where": "backend/eval/splits.py load_split/assert_disjoint",
    "shapeOrFormat": "JSON list of entries each with match_id (or id fallback) + split in
{'train','eval'}; raises on overlap. Pure-logic guard, not yet wired into

```

```

distill_local/run_regression (wiring deferred per its docstring)."
    }
],
    "videoDependency": "SCORING and the existing experiment/ablation engines are fully video-free. metrics.py, segmentation_metrics.py, gt_loader.py, splits.py, regression.py (the pure parts) need no video/GPU/numpy. experiment.compare and ablation.run_ablation run end-to-end on CPU from in-memory VideoCandidates / committed-shaped *_golden_features.json ? no video, no DB, no Gemini. The DB-read path (harness.get_predictions, regression.regress_with_provider, run_eval/run_regression) is video-free at run time BUT depends on a SQLite DB that only the GPU pipeline (run on real video) can populate ? so without a committed file it cannot run in CI. The FEATURE-PRODUCING step (fusion_golden person detection over the original MP4) and the served-path windowing (served_gate_a reads cached trajectory CSVs, GPU-free with person cue OFF) need the out-of-repo golden corpus (trajectories + rallies.csv via the gitignored manifest). Net: the design's goal is achievable with ZERO video by committing a frozen predictions/feature substrate + labels and feeding the already-pure scoring/regression layer.",
    "builtVsPlanned": "BUILT & shipped (with passing pytest): pure metrics + segmentation metrics; canonical GT loader (gt_loader, closes the two-loader fork, ADR-008 prereq); DB-read harness + run_eval CLI; baseline-snapshot regression gate + run_regression CLI with exit 0/1/3 and incommensurability guards; experiment A/B engine (compare/evaluate_variant/compare_unlabeled/record_experiment) with committed leaderboard; LOSO/LOGO ablation driver with honest CI+sign-test verdicts (earns_keep/suggestive/inert/structural_protected) + PDF export; fusion_golden feature extractor (with run-telemetry black box); export_wasb_segments for eyeball verification; served_gate_a production-path litmus; calibrate_wasb + distill_local; splits train/eval guard (pure-logic). Committed fixtures: eval_baselines/{heuristic_lovo_n6.json, experiment_leaderboard.json, gemini_wrapper_2026-06-10.json}. PLANNED / NOT built: a committed regression_baseline.json (default path reserved, file absent); committed golden label CSVs / trajectories / *_golden_features.json (all currently land in gitignored output/ or outside the repo); splits wiring into distill_local/run_regression (explicitly deferred in its docstring); regression as a CI gate (CI runs only lint/mypy/import-smoke/pytest+coverage). No 'video-free regression from committed structured files' runner exists yet ? the engine exists, the committed inputs and the file-fed predictions loader do not.",
    "constraints": [
        "Commercial-clean licensing: every dependency code/data/weights MIT/Apache-2.0/BSD only; paid LLMs are inference/serving/fallback only, never a training data source.",
        "Dev container lacks GPU/torch/cv2/numpy ? the full detect pipeline cannot run here; pure-logic eval (metrics, gt_loader, regression core, experiment, ablation, splits) DOES run here. A committed file-driven regression must stay in the pure-logic lane to be CI-runnable.",
        "output/ is gitignored (.gitignore line 39) and *.db is gitignored (lines 36-38) ? so the runtime DB, the manifest, golden_features JSON, and the default regression_baseline.json location do NOT survive a clean checkout unless explicitly relocated. eval_baselines/ is the established TRACKED home for committed eval artifacts.",
        "Honest small-n contract (owner directive 'NUMBERS INFORM, FOUNDATIONS DECIDE'): never a bare mean ? every signal/regression delta ships a bootstrap CI + exact paired sign test (eval_stats); weak/non-significant cues are retained default-OFF, never deleted on a small-n null; structural guards (Gate A) are never auto-demoted on 'no lift'.",
        "Train/eval separation: split unit = match id (never clip/MD5); splits.assert_disjoint must guard any fixture used both for training a scorer and for regression baselines.",
        "gt_hash / label_set_hash / spec_hash fingerprints must travel with any committed baseline so a comparison against changed labels or a changed recipe is flagged incommensurable rather than silently trusted (the existing regression.py and experiment.py guard contract).",
        "eval must not drift from serve: distill_local.PROPOSAL vs SERVED windowing are single-sourced in backend.eval.windowing; served_gate_a pins production operating point (vel 0.02, merge 2.0, min 2.0). A file-driven golden must record which windowing/operating-point it was frozen at."
    ],
    "relevantPaths": [
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/segmentation_metrics.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gt_loader.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/harness.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py",
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/ablation.py",

```

```

"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/fusion_golden.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/export_wasb_segments.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/served_gate_a.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/splits.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/run_eval.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/heuristic_lovo_n6.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/experiment_leaderboard.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/gemini_wrapper_2026-06-10.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_gt_loader.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_harness.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_fusion_golden_telemetry.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_served_gate_a_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_export_wasb_segments.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/.github/workflows/ci.yml",
"C:/Users/avidu/Projects/badminton-highlight-indexer/.gitignore"
],
"risksGaps": [
  "No committed golden fixture survives a clean checkout for an END-TO-END regression: the n=6 corpus (trajectories + .rallies.csv) is referenced only by the gitignored output/human_lovo_manifest.json via absolute, machine-specific Windows paths; the *_golden_features.json substrate lands in gitignored output/. So a 'video-free regression from committed files' must FIRST commit a (small) frozen substrate + labels + a repo-relative manifest. eval_baselines/ is the proven tracked home.",
  "regression.regress can only obtain PREDICTIONS via harness.get_predictions, which queries the SQLite DB. There is no existing path to feed regression committed predictions from a file ? a thin 'predictions-from-file' loader (or reusing experiment.VideoCandidates -> evaluate_variant) must be ADDED so the gate runs with no DB.",
  "The two truth-sources can diverge: harness/regression score DB rows (candidate_segments / play_segments) while experiment/ablation/served_gate_a score trajectory-derived windows. A file-driven design should pick ONE substrate and record its provenance (windowing/operating-point, detector, gt_hash) to avoid an incommensurable committed baseline.",
  "Regression is NOT a CI gate today (ci.yml runs only lint/mypy/import-smoke/pytest+coverage). If the design intends CI enforcement, that wiring is net-new and must run in the no-GPU lane.",
  "splits.assert_disjoint exists but is NOT wired into regression/distill_local ? committing a golden fixture risks silent train/eval leakage unless the guard is connected when the fixture is introduced.",
  "test_served_gate_a_regression.py and the golden-corpus tests skip cleanly when files are absent; this means the production-path Gate-A litmus is effectively UNVERIFIED in CI ? a committed mini-substrate would also let that litmus (or a reduced form) actually run in CI.",
  "A committed golden corpus must respect the licensing + minors-exclusion guardrails; raw match footage cannot be committed, but derived trajectory CSVs / feature JSON / interval labels are the safe, video-free artifacts to commit."
]
},
{
  "area": "Golden-set storage, sharing, and regression model ? design + built state",
  "summary": "Golden ground truth is per-video rally [start,end) interval CSVs, keyed by file MD5 (video_id), produced by the Roora vendor (schema v8) and ingested via the sibling collector. The eval harness (metrics/regression/experiment) is structurally video-free already ? it re-scores predictions stored in SQLite and golden labels read from CSV. CRUX: NO doc proposes committing structured per-video label or feature artifacts to the repo for video-free regression. The deliberate design is the OPPOSITE: GOLDEN_DATA_SHARING.md keeps the heavy per-video artifacts (golden-label CSVs, *_golden_features.json, trajectory CSVs, manifests) OUT of

```

git ? they live in gitignored output/ and sync between the GPU box and CPU dev sessions via a PRIVATE OneDrive folder, explicitly because they derive from the proprietary corpus and must never be committed. What IS tracked in git is only the small SCORE artifacts in eval_baselines/ (aggregate + per-video F1, deltas, honest CI/sign-test, label_set_hash, spec hashes) ? no raw labels or features. So a new proposal to commit structured per-video artifacts for video-free regression would EXTEND a gap, not duplicate anything; it must reconcile with (a) the OneDrive sharing convention and (b) the licensing/privacy guardrail that keeps proprietary-derived per-video data private.",

"keyFindings": [

"(1) HOW STORED/SHARED TODAY: Golden labels = one CSV per video, one rally per row, [start,end). Two schema layers: the minimal eval-harness format (EVALUATION.md §2: 'start,end' header optional; decimal seconds OR MM:SS/HH:MM:SS; malformed rows fail loudly) read by backend/eval/annotations.py + gt_loader.load_gt_intervals (the canonical golden reader). The richer VENDOR deliverable (ROORA_LABELING_GUIDELINES.md §1) is a per-rally CSV named exactly <video_id>.csv. Keying is by video_id = file MD5 (CLAUDE.md; EVALUATION.md §3; compute_video_id) ? the vendor is told NOT to trim/re-encode because the pipeline IDs by checksum. The golden-label naming convention in code is X.rallies.csv -> X.MP4 (fusion_golden.py::_derive_video_path).",

"(1b) LOCATION: live golden CSVs + the GPU-extracted *_golden_features.json + trajectory CSVs live in gitignored output/ (per-machine) and scattered local folders.

GOLDEN_DATA_SHARING.md (#175, 2026-06-15) wires GPU box <-> CPU dev sessions via a single PRIVATE OneDrive folder: %USERPROFILE%/OneDrive/rally-annotator/golden-data/ with subdirs features/<corpus-tag>/, trajectories/<corpus-tag>/, manifests/, baselines/. corpus-tag = e.g. 2026-06-15-n6. Raw videos are NEVER put there (kept local to GPU box). Optional ergonomics: RALLY_GOLDEN_DIR env var + scripts/sync_golden.py (in #175, not required).",

"(2) CRUX ? NO doc proposes committing structured per-video LABEL/FEATURE artifacts to the repo for video-free regression. The codebase ALREADY supports video-free regression a different way: detection runs ONCE per video, all per-cue signals + per-candidate features persist to SQLite candidate_segments.stats (JSON) and to output/*_golden_features.json, then ANY variant re-scores OFFLINE on CPU with no GPU/video (EXPERIMENT_HARNESS.md §2, §5.1; ABLATION_LAYER.md; standing lesson 'persist detection once, re-score forever'). But the persistence target is gitignored output/ + private OneDrive, NOT the repo. GOLDEN_DATA_SHARING.md is explicit: 'The shared folder lives OUTSIDE the repo -> never committed. Keep output/ gitignored... features derive from the Proprietary corpus -> keep the OneDrive folder private.' So committing per-video artifacts to git would be a NEW extension, contradicting the current private/OneDrive sharing posture unless reconciled.",

"(2b) WHAT IS COMMITTED TO THE REPO TODAY (verified via git ls-files): only eval_baselines/*.json SCORE artifacts ? experiment_leaderboard.json (per-A/B-comparison aggregate P/R/F1/mIoU + honest_delta_f1 CI + sign test + label_set_hash + variant spec_hash + promotion/revert records), heuristic_lovo_n6.json (mean + per_video F1 for the 6-match corpus), gemini_wrapper_2026-06-10.json. These contain SCORES and hashes, NOT raw rally labels or feature values. regression.py's DEFAULT_BASELINE_PATH eval_baselines/regression_baseline.json is referenced but NOT present/tracked. So the repo already commits a fingerprint+score layer (label_set_hash, gt_hash/gt_fingerprint) that ties a baseline to a label set WITHOUT storing the labels ? a natural anchor any new per-video-artifact proposal should build on.",

"(3) SCHEMA + VERSIONING (Roora): ROORA_LABELING_GUIDELINES.md is at VERSION v8 (2026-06-13), declared the SINGLE canonical copy and the sole label to use going forward; it supersedes Drive copies v3/v5/v6/v7 and an internal 'DRAFT v2' (those numbers never matched). The in-repo file is canonical because the fusion fields originate here; a sibling-collector mirror, if staged, must be regenerated from this file. Columns: rally_number, start_mmss, end_mmss, start_time(sec, auto), end_time(sec, auto), shots_count, ending_reason {winner|forced_error|unforced_error|service_fault|let|other}, last_shot_outcome {in|net|out|n_a}, score_before, score_after, notes. Plus a fusion-added per-video INTAKE MANIFEST (NOT in the per-rally CSV): video_id, fps, court_corners (4 px points), net_line (2 px endpoints), singles_or_doubles. ending_reason is shared across all net-separated sports. Three golden rules: ignore all dead time; include EVERY rally (incl. service faults/lets, EXCEPT warm-up and partial edge rallies); a shot = one contact. Quality bar: boundaries within $\pm 0.3s$, frame-grid boundaries auto-FAIL. Tier-B (per-shot timestamps, per-shot type) is appendix-only, do-not-start.",

"(4) BUILT vs PLANNED vs OWNER-GATED. BUILT: GS-1 FileProvider + AnnotationProvider registry (backend/annotations/, providers file+auto); GS-2 regression.regress()/regress_with_provider + save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle scaffold; the full eval engine (metrics.py, calibration.py LOVO, classifier.py pinnable JSON model, training.py label/feature glue); EXPERIMENT_HARNESS P1 experiment.py compare()/compare_unlabeled()/record_experiment (shipped

2026-06-13); ABLATION_LAYER Phase-0 ablation.py (LOSO/LOGO, CI+sign test, litmus test); fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus; collector import-local for 3 sports; Roora v8 guideline. PLANNED/owner-gated: GS-3 HumanVendorProvider (parked on Roora pilot confirming quality); GS-5 trigger wiring (CI gate/scheduled); EXPERIMENT_HARNESS P2 per-cue flags+feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (design only, not started); internal gold key still pending. GATE-A: PROMOTED then REVERTED (#146->#151, eval!=serve) ? now OFF-by-default, opt-in only.",

"(5) STATED CONSTRAINTS ON SHARING GOLDEN DATA. Licensing/privacy: golden sets built ONLY from licensed/owned footage (founder's collection + ShuttleSet); NO end-user uploads ever flow to a vendor; all third-party video egress through ONE provider boundary with DPA/no-reuse/delete-on-delivery/access-logging (GOLDEN_SET_VENDORING.md §2,§4). Repo: per-video golden artifacts are proprietary-derived -> kept OUT of git, private OneDrive only, no public share links (GOLDEN_DATA_SHARING.md Guardrails). Minors: EXCLUDED ENTIRELY from the labeled pool ? footage with children is not annotated, flagged back; minors-exclusion enforced at the pooling query (ROORA v8 §8 data-handling; QUALITY_ITERATIONS personalization decisions; CLAUDE.md). Biometrics/identity: no biometric/identity labeling, players referred to by per-video pseudonyms only, no cross-video person ID, no gait, no pose/skeleton/keypoint; work-for-hire IP assignment of labels (ROORA v8 §8). Train/eval leakage: split by match/session never by clip; hold out whole conditions; train+eval in separate catalogs with HARD ingest/export guardrails still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)."

```

],
"dataStructures": [
  {
    "name": "Golden rally label CSV (eval-harness minimal format)",
    "where": "docs/EVALUATION.md §2; read by backend/eval/annotations.py +
gt_loader.load_gt_intervals; naming X.rallies.csv (fusion_golden.py)",
    "shapeOrFormat": "One CSV per video. Header 'start,end' optional/auto-detected. One
rally per row: start,end as decimal seconds (102.5) OR MM:SS/HH:MM:SS (00:01:12), forms mixable.
Blank lines and # comments ignored. Malformed rows (bad ts, start>=end) fail loudly. video keyed
by file MD5."
  },
  {
    "name": "Roora vendor per-rally CSV (schema v8)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §1; ingested via sibling collector python -m
src.cli.curate import-local",
    "shapeOrFormat": "One CSV per video named <video_id>.csv, UTF-8, header required, one
row per rally chronological. Cols: rally_number(int 1-based), start_mmss(m:ss.mmm), end_mmss,
start_time(sec auto), end_time(sec auto), shots_count(int),
ending_reason(winner|forced_error|unforced_error|service_fault|let|other),
last_shot_outcome(in|net|out|n_a), score_before, score_after, notes(required when other). Rules:
end>start; no overlaps end[N]<=start[N+1]; ±0.3s boundary tolerance."
  },
  {
    "name": "Per-video intake manifest (fusion-added)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §A; template INTAKE_MANIFEST_TEMPLATE.csv
(collector side)",
    "shapeOrFormat": "One row per video, recorded once (NOT in the per-rally CSV):
video_id(str), fps(num), court_corners(4 (x,y) px points clockwise from near-left), net_line(2
(x,y) px endpoints), singles_or_doubles(singles|doubles). Worked ex:
court_corners=[(360,980),(1560,980),(1330,300),(590,300)], net_line=[(470,640),(1450,640)]."
  },
  {
    "name": " *_golden_features.json (replayable offline re-scoring substrate)",
    "where": "written by backend/eval/fusion_golden.py to gitignored
output/<name>_golden_features.json; shared via OneDrive features/<corpus-tag>; NOT committed to
git",
    "shapeOrFormat": "Per-video JSON record: {name, fps, frame_width, candidates:[{start,
end, shuttle{5 fps/res-invariant feats:
duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person{5 feats:
players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
label(int via IoU match to GT)}], gts}. Re-scored forever on CPU by fusion_compare/ablation ? no
GPU/video."
  },
  {
    "name": "candidate_segments table (SQLite persistence substrate)",

```

```

    "where": "backend/infrastructure/database.py (add_candidate_segment :318,
query_candidate_segments :355); output/<db_name>",
    "shapeOrFormat": "Per-candidate row: video_id, detector, start_time, end_time,
chunk_index, confidence, stats(JSON ? per-cue features), detection_params(JSON), human_label,
confirmed_segment_id. The persist-once/re-score-forever substrate; keyed by video_id (file
MD5).",
  },
  {
    "name": "eval_baselines/experiment_leaderboard.json (TRACKED in git ? scores only)",
    "where": "eval_baselines/experiment_leaderboard.json; written by
experiment.record_experiment",
    "shapeOrFormat": "JSON array of comparison/promotion records: {timestamp, iou,
label_set_hash, num_videos,
variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta:{...}, honest_delta
_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test:{wins,losses,ties,n_effective,p_valu
e}}, optional promotion:{cue,promoted_to,decision,reverted,...}}. Contains SCORES + hashes, NO
raw labels/features."
  },
  {
    "name": "regression baseline.json (per-video score snapshot)",
    "where": "backend/eval/regression.py DEFAULT_BASELINE_PATH =
eval_baselines/regression_baseline.json (referenced; not currently tracked);
eval_baselines/heuristic_lovo_n6.json is a tracked sibling",
    "shapeOrFormat": "save_baseline(video_id, stage, precision, recall, f1, ...,
gt_fingerprint). RegressionResult: regressed, reasons, gt_hash, baseline P/R, tolerance,
has_baseline. heuristic_lovo_n6.json: {name, description, computed, mean_f1, std,
per_video:{<video_name>: f1}}. Per-video keyed by video_id/name; ties a baseline to a label set
via gt_fingerprint/label_set_hash WITHOUT storing labels."
  }
],
"videoDependency": "The eval/regression machinery is ALREADY video-free at score time and
that is the explicit architecture: detection (GPU/WSL, needs the video) runs ONCE per video; all
per-cue signals + per-candidate features persist (SQLite candidate_segments.stats JSON and
output/*_golden_features.json); then fusion_compare.py / ablation.py / experiment.compare() /
run_regression.py re-score ANY number of variants on pure CPU reading only the small JSON + the
golden-label CSV ? no video, no GPU, no API calls, deterministic (EXPERIMENT_HARNESS $2,
ABLATION_LAYER, EVALUATION $intro). Golden-label CSVs themselves require no video to consume.
What is NOT yet video-free / not in the repo: the per-video artifacts (golden-label CSVs,
*_golden_features.json, trajectory CSVs, manifests) are gitignored and shared only via private
OneDrive (GOLDEN_DATA_SHARING.md), so a fresh git clone alone CANNOT run a regression ? it has
the score baselines (eval_baselines/) and the harness code but not the labels/features. A clone
CAN run the deterministic synthetic-row unit tests; the golden-litmus tests
(test_ablation.py::test_litmus_reproduces_gate_a, test_fusion_compare.py) skipif the
output/*_golden_features.json substrate is absent. So committing structured per-video artifacts
to the repo is exactly what would make regression runnable from a clone alone ? the unfilled
gap.",
"builtVsPlanned": "BUILT (shipped): GS-1 AnnotationProvider ABC+registry+FileProvider
(backend/annotations/, keys file+auto); GS-2 regression.regress()/regress_with_provider +
save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle
scaffold; eval engine
metrics.py/calibration.py(LOVO,cross_validate,improvement_report)/classifier.py(pinnable JSON
LogisticRegression)/training.py(label_candidates,build_training_set); EXPERIMENT_HARNESS P1
experiment.py compare()/compare_unlabeled()/record_experiment + JSON leaderboard (2026-06-13);
honest-stats helpers C1-C4 (eval_stats.py, score_calibration.py, segmentation_metrics.py ?
landed, NOT yet wired into default harness flow); ABLATION_LAYER Phase-0 ablation.py with litmus
test; fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus + test;
FusionSegmenter (fusion_hybrid) default-OFF; collector import-local for 3 sports; Roora v8
guideline (in-repo canonical). PLANNED (not started): GS-3 HumanVendorProvider (parked until
Roora pilot confirms quality); GS-5 trigger wiring; EXPERIMENT_HARNESS P2 per-cue enable flags +
feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (L0-L3, design only);
internal gold key; hard train/eval-catalog ingest/export guardrails (BACKLOG);
GOLDEN_DATA_SHARING optional RALLY_GOLDEN_DIR + sync_golden.py. OWNER-GATED / NOTABLE: GATE-A
net_crossings>=2 was PROMOTED to served default (#146) then REVERTED (#151) as an eval!=serve
regression (offline +0.074 on proposal windowing did NOT transfer to coarse served windowing,
mean served ?F1 -0.008) ? now OFF-by-default, opt-in (indexing.fusion.min_crossings=2). Person

```


cue ?F1 -0.021 and +audio +0.079 both fail the promotion bar at n=6 (CIs span 0) -> stay default-OFF, retained as feature columns. Whole golden-set/owned-model implementation track is owner-gated; corpus = 6 labelled matches in output/human_lovo_manifest.json.",

"constraints": [

"Golden data built ONLY from licensed/owned footage (founder collection + ShuttleSet); NO end-user uploads ever flow to a vendor (GOLDEN_SET_VENDORING §2)",

"Per-video golden artifacts are proprietary-derived -> kept OUT of git; shared only via PRIVATE OneDrive (no public links); output/ stays gitignored; do not symlink the shared folder into a tracked path (GOLDEN_DATA_SHARING Guardrails)",

"Only eval_baselines/*.json SCORE artifacts (aggregate+per-video F1, deltas, CI/sign-test, label_set_hash, spec hashes) are committed to the repo ? NOT raw labels or feature values",

"Minors EXCLUDED ENTIRELY from the labeled/training pool; footage with children is not annotated and flagged back; minors-exclusion enforced at the pooling query (ROORA v8 §8; CLAUDE.md GDPR Art.9)",

"No biometric/identity/gait/pose/skeleton/keypoint labeling; players referenced by per-video pseudonyms only, no cross-video person ID; labels are work-for-hire IP-assigned (ROORA v8 §8)",

"Third-party video egress through ONE provider boundary with DPA, no-reuse, delete-on-delivery, access logging; NDA before any footage shared (GOLDEN_SET_VENDORING §4; ROORA §8)",

"video_id = file MD5 is the immutable key ? vendor must NOT trim/re-encode/rename source video or the checksum (hence video_id) changes (ROORA v8 §1; EVALUATION §3)",

"Train/eval split by match/recording-session never by clip; hold out whole conditions; broadcast minimal in eval; HARD ingest/export guardrails so a video can't land in both catalogs are still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)",

"Honest small-N contract: never a bare mean ? promote a cue only when ?F1 CI excludes 0 AND the sign test agrees; structural guards (Gate-A) exempt from auto-demotion (ABLATION_LAYER; QUALITY_ITERATIONS)",

"Experiment infra is single-tenant/local-first: a JSON leaderboard the size of baseline.json ? explicitly NOT MLflow/W&B/a service (EXPERIMENT_HARNESS §9)",

"Roora v8 is the SINGLE canonical guideline (supersedes Drive v3/v5/v6/v7 + internal DRAFT v2); the in-repo file is canonical, any collector mirror regenerated from it (ROORA header)",

"Fast-tier AutomatedProvider oracle MUST be a different model lineage than production (else shared blind spots -> false green); a fast-tier regression is a smoke alarm that escalates to the human tier (GOLDEN_SET_VENDORING §5; GS-4)"

],

"relevantPaths": [

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_IMPLEMENTATION.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EVALUATION.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EXPERIMENT_HARNESS.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ABLATION_LAYER.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ROORA_LABELING_GUIDELINES.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\QUALITY_ITERATIONS.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PER_VIDEO_OUTPUT_LAYOUT.md",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\experiment_leaderboard.json",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\heuristic_lovo_n6.json",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\gemini_wrapper_2026-06-10.json",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\experiment.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\ablation.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\annotations\\base.py",

"C:\\Users\\avidu\\Projects\\badminton-highlight-

```

indexer\\backend\\infrastructure\\database.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_regression.py"
],
"risksGaps": [
    "CRUX RESTATED for the caller: do NOT assume an existing 'commit per-video artifacts to repo' proposal exists ? there is none. The only existing per-video-artifact sharing design is GOLDEN_DATA_SHARING.md, which deliberately keeps them OUT of git (private OneDrive). A new proposal would EXTEND this gap but must not CONTRADICT: (a) the 'never commit proprietary-derived data' guardrail, (b) the OneDrive sync convention, (c) the minors/biometric exclusions. Likely reconciliation: commit only DERIVED, de-identified, label-free or licensing-clean artifacts (e.g. the *_golden_features.json feature vectors are scale/translation-invariant counts/ratios, not raw frames or coords ? these MAY be repo-safe where raw CSVs/videos are not; confirm with owner).",
    "The repo already commits a fingerprint layer (label_set_hash in experiment_leaderboard.json; gt_fingerprint/gt_hash in regression.py) that binds a score baseline to a label set without storing labels. Any new per-video-artifact-in-repo design should build on these hashes for integrity/keying rather than inventing a parallel scheme.",
    "regression.py DEFAULT_BASELINE_PATH points at eval_baselines/regression_baseline.json but that file is NOT present/tracked ? run_regression.py would hit exit-3 (no-baseline) on a fresh clone. The tracked baselines are experiment_leaderboard.json + heuristic_lovo_n6.json + gemini_wrapper_*.json (different shape). A clone cannot currently run the per-video regression gate end-to-end.",
    "Golden-litmus tests skipif the output/*_golden_features.json substrate is absent ? so on CI / a fresh clone the golden-litmus coverage is silently skipped; only synthetic-row unit tests run. Committing the (de-identified) feature substrate would also light up these tests in CI ? a concrete motivation for the proposal.",
    "Two schema layers coexist (minimal 'start,end' eval CSV vs the rich Roora <video_id>.csv). The collector ingests the rich one; the indexer harness reads the minimal one via gt_loader. Any artifact-in-repo proposal must be explicit about which schema it freezes and how the rich->minimal projection is captured.",
    "Train/eval leakage hard-guardrails (separate catalogs enforced at ingest/export) are still only a convention + BACKLOG TODO ? a video could currently land in both train and eval by mistake; relevant if committed artifacts blur the train/eval boundary.",
    "Gate-A eval!=serve revert is the cautionary tale baked into the leaderboard: a number measured on the harness windowing did not transfer to the served windowing. Any regression model committed to the repo must record WHICH windowing/operating-point the baseline was measured at, or it will mispredict served behaviour."
]
},
{
    "area": "Pipeline data contracts and video-free seam(s) for regression",
    "summary": "The pipeline is a clean 4-stage chain: (1) DETECTION ? the WSL shuttle detector (WASB/TrackNet) reads frames and emits a per-frame trajectory CSV (`Frame,Visibility,X,Y`); (2) WINDOWING ? `TrackNetRunner.trajectory_to_action_windows` (the model-agnostic `\"brain\"`) turns the trajectory into candidate rally window dicts, persisted to `candidate_segments`; (3) ADJUDICATION/HANDOFF ? fusion gates + AI provider confirm windows into `play_segments`; (4) STITCH ? `VideoCompiler.compile_highlights` re-encodes/concats the selected `(start,end)` time-pairs into the reel. Video frames are required in exactly TWO places: the WASB/TrackNet trajectory pass (Stage 1) and the person-detector pass (fusion P2, default-OFF) ? both consume the original video by absolute frame index. EVERYTHING from the trajectory CSV onward (windowing, net-crossing Gate A, person features given boxes, candidate persistence, IoU scoring, segment selection, merge) is PURE deterministic transformation. The provider handoff (Gemini) and the final ffmpeg stitch are the only other steps that touch the media file. The single best video-free regression seam is the TRAJECTORY CSV: freeze `<key>wasb.csv` per video and the windowing?gating?scoring chain replays with no GPU/WSL/frames. A second, complementary seam is the persisted `candidate_segments` rows (post-windowing), and for fusion the per-frame person-detection dict.",
    "keyFindings": [
        "STAGE SEQUENCE (runtime, trajectory hybrids = the production shuttle path): config load -> video_id=MD5(file) (backend/utils/hashing.py::compute_video_id, 64KB chunks) -> validators -> db.add_video (UPSERT) -> segmenter.process_video. Inside TrajectoryHybridSegmenter.process_video: P1 healthcheck gate -> P2 runner.run_predict(video,out_dir='output/<family>',video_id) writes trajectory CSV; parse_trajectory_csv -> List[TrajectoryPoint]; trajectory_to_action_windows -> candidate window dicts; _enrich_candidate_stats per window -> window_stats; db.add_candidate_segment persists
    ]
}

```

```

each (full superset) -> P3 _select_for_handoff picks indices; for each,
extract_video_chunk(reencode=True) padded clip -> provider.analyze_video(clip,prompt) -> AI
segment dicts (or skip_ai => windows emitted verbatim) -> P4 db.add_play_segment +
link_candidate_to_segment. Then COMPILE (separate call): db.query_play_segments -> filter by
stitching.min_shot_count -> time_pairs=[(start_time,end_time)] ->
VideoCompiler.compile_highlights(filepath,time_pairs,out_name).",
"ARTIFACT 1 ? Trajectory CSV (Stage-1 output, the natural freeze seam): file
`output/<family>/<stem>__<video_id[:12]>_wasb.csv` (WASB) or `<key>_predict.csv` (TrackNet).
Exact columns: `Frame,Visibility,X,Y` (header + order fixed; Visibility==1 means visible; X,Y
are ORIGINAL-frame pixel coords). Parsed to TrajectoryPoint{frame:int, visible:bool, x:float,
y:float}, frame-sorted. This is the complete handoff from the GPU/WSL world to the pure-Python
world.",
"ARTIFACT 2 ? Candidate window dict (output of trajectory_to_action_windows; in-memory):
{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int (==1 for shuttle trajectory), chunk_index:Optional[int]
(==0 for whole-video trajectory pass)}. Windowing is deterministic: active frame iff visible AND
normalized per-second velocity (dist/frame_width * fps/dframes) > velocity_thresh; active frames
within merge_gap*fps grouped; windows shorter than min_window_duration dropped. fps<=0 ->30.0
and frame_width<=0 ->1280.0 silent fallbacks. visible==False resets prev (occlusion breaks the
track).",
"ARTIFACT 3 ? candidate_segments DB row (persisted, detector-agnostic fine-tuning table;
THE second freeze seam): columns id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL(=end-start), confidence:REAL(=min(1.0,peak_velocity)), stats:JSON,
detection_params:JSON, confirmed_segment_id(FK SET NULL), human_label, notes, created_at,
user_id(NULL=local). stats JSON (base trajectory) = {peak_velocity, mean_velocity,
active_frame_count, track_id_count}. detection_params by detector: wasb={detector,velocity_thres
h,merge_gap,min_action_window_duration,weights_path,fusion_substrate?}; tracknet adds
queue_length; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_t
hreshold,min_action_window_duration}.",
"PER-SIGNAL FEATURE COLUMNS ? Gate A (net-crossings):
FusionSegmenter._enrich_candidate_stats ALWAYS adds stats['net_crossings']:float via
rally_gate.gate_windows (pure, from the trajectory; axis/net estimated once per video, cached).
Person cue (default-OFF, only if indexing.fusion.cue_flags.person): adds the 5
PERSON_FEATURE_NAMES = players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all scale/translation-invariant floats from
fusion_features.compute_person_features). Court mask = optional normalized
indexing.fusion.court_polygon; net=(net_axis,net_position). The learned-classifier shuttle
features (distill_local.FEATURE_NAMES) = duration, peak_velocity, mean_velocity, active_ratio,
net_crossings; the Gen-0 per-frame schema (backend/features/rally_seq.py FEAT_NAMES) = vis_rate,
spd_mean, spd_max, cross_rate, x_std, y_std.",
"ARTIFACT 4 ? play_segment dict (final segment record, db.add_play_segment /
query_play_segments): {id:int, video_id, start_time:float, end_time:float(=end-
start), server, receiver, metadata:dict, events:[...]}. Hybrid P4 metadata = {shot_count:int
(provider shuttle_exchange_count else max(3,int(dur/0.8))), shot_count_confidence:str (provider
value or 'Unknown'/'LocalNoAI'), detector:f'{family}-hybrid-{model_name}'}. Provider (AI)
segment dict consumed by P3 = {start_time, end_time, shuttle_exchange_count?,
shot_count_confidence?} (timestamps offset by clip w_start; _candidate_id stamped internally).",
"FINAL REEL transform (VideoCompiler.compile_highlights): segments=[(start,end)] ->
_merge_overlapping (parse times, drop end<=start, sort, merge overlapping/abutting ranges so a
rally is never stitched twice) -> per-clip ffmpeg re-encode (libx264/superfast/crf22, identical
settings, optional watermark via -vf) -> `-f concat -c copy` -> output/<name>.mp4. Selection
upstream: API filters by stitching.min_shot_count (segments without shot_count are exempt) and
by req.segment_ids, then time_pairs=[(s['start_time'],s['end_time'])].",
"TWO ALTERNATE PATHS that bypass the trajectory seam: `gemini` segmenter (pure-AI, no
P2/candidates ? chunks the whole video, uploads, provider returns segments directly to
play_segments) and `motion` (pixel-absdiff baseline with FABRICATED random metadata/events).
These two are the _FINAL_ONLY_SEGMENTERS in eval/batch.py ? they have no candidate stage and
cannot be frozen at the trajectory seam. The video-free seam analysis applies to the trajectory
hybrids (wasb_hybrid/tracknet_hybrid/fusion_hybrid), which are the rally-quality track."
],
"dataStructures": [
{
"name": "Trajectory CSV",
"where": "output/<family>/<stem>__<video_id[:12]>_wasb.csv (WASB) / <key>_predict.csv
(TrackNet); written in WSL, read by

```

```

backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` (fixed order); rows: Frame:int,
Visibility:int(1=visible), X:float px, Y:float px ->
TrajectoryPoint{frame:int,visible:bool,x:float,y:float} frame-sorted"
},
{
    "name": "Candidate window dict",
    "where": "output of TrackNetRunner.trajectory_to_action_windows (tracknet_runner.py);
in-memory, consumed by trajectory_hybrid P2/P3",
    "shapeOrFormat": "{start:float(sec), end:float(sec), active_frame_count:int,
peak_velocity:float, mean_velocity:float, track_id_count:int(==1),
chunk_index:Optional[int](==0)}"
},
{
    "name": "candidate_segments row",
    "where": "backend/infrastructure/database.py (add_candidate_segment /
query_candidate_segments), table candidate_segments",
    "shapeOrFormat": "id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count[,net_crossings,+5
person feats]}, detection_params:JSON, confirmed_segment_id, human_label, notes, created_at,
user_id"
},
{
    "name": "Per-signal feature columns (fusion)",
    "where": "backend/pipeline/detectors/fusion_features.py::PERSON_FEATURE_NAMES +
rally_gate net_crossings; persisted into candidate_segments.stats by
fusion_hybrid.py::enrich_candidate_stats",
    "shapeOrFormat": "net_crossings:float (Gate A, always); person (flag-gated):
players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all floats, scale/translation-invariant)"
},
{
    "name": "Person per-frame detections",
    "where": "backend/pipeline/detectors/person_detector.py::detections_per_frame (fusion
P2, needs video+GPU)",
    "shapeOrFormat": "{ 'frames': {frame_idx:int -> [(track_id:int, bbox:(x1,y1,x2,y2)),
...]}, 'fps':float, 'frame_width':float, 'frame_height':float}; frame_idx aligns 1:1 with
trajectory .frame"
},
{
    "name": "play_segment dict (final)",
    "where": "backend/infrastructure/database.py::add_play_segment / query_play_segments,
table play_segments",
    "shapeOrFormat": "{id:int, video_id, start_time:float, end_time:float,
duration:float(=end-start), server, receiver,
metadata:dict{shot_count:int,shot_count_confidence:str,detector:'<family>-hybrid-<model>'},
events:[...]}"
},
{
    "name": "AI/provider segment dict",
    "where": "provider.analyze_video output, consumed by trajectory_hybrid P3 and
gemini.py",
    "shapeOrFormat": "{start_time:float(decimal sec), end_time:float,
shuttle_exchange_count?:int, shot_count_confidence?:'Low'|'Medium'|'High'}; +_candidate_id
stamped internally; times offset by clip start"
},
{
    "name": "Final reel time_pairs",
    "where": "backend/main.py::compile_highlights -> VideoCompiler.compile_highlights
(compiler.py)",
    "shapeOrFormat": "List[Tuple[start:float, end:float]] -> _merge_overlapping (sort+merge)
-> per-clip reencode + concat -c copy -> output/<name>.mp4"
},
{

```

```

    "name": "GT / golden CSV (eval comparison)",
    "where": "backend/eval/annotations.py::load_annotations (start,end) and
calibrate_wasb.load_golden_gts (rally_number,start_time,end_time)",
    "shapeOrFormat": "generic: `start,end` 2-col (RAISES on bad row); golden/slicer:
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]` (SILENTLY skips bad rows)"
  }
],
  "videoDependency": "Video/frames are REQUIRED in exactly two compute steps, both at the
front of the trajectory-hybrid pipeline: (A) the shuttle trajectory pass ?
WasbRunner/TrackNetRunner.run_predict stages the video into WSL and runs the GPU CNN over
decoded frames to emit the `Frame,Visibility,X,Y` CSV (backend/pipeline/detectors/wasb_runner.py
+ wasb_infer.py, GPU/WSL/cv2); and (B) the person-detector pass ?
PersonDetector.detections_per_frame seeks the ORIGINAL video by absolute frame range (fusion P2,
default-OFF, torch/cv2/GPU). In addition the media file is touched (not 'frames' analysis but
read+transcode) at the AI handoff (extract_video_chunk re-encodes a padded clip and uploads to
Gemini) and at the final stitch (VideoCompiler re-encodes/concats clips with ffmpeg). EVERYTHING
ELSE is pure deterministic Python on numbers/strings: parse_trajectory_csv,
trajectory_to_action_windows (windowing brain), rally_gate (net-crossings Gate A),
fusion_features (person features given boxes ? pure, no cv2/torch), db.add_candidate_segment /
add_play_segment persistence, eval/metrics IoU scoring, _select_for_handoff gating, and
_merge_overlapping. So once the trajectory CSV (and, for fusion, the per-frame person dict)
exists, no video, GPU, WSL, ffmpeg, or cloud is needed to reproduce windowing, gating, candidate
rows, and IoU scores.",
  "builtVsPlanned": "BUILT and live: full trajectory-hybrid path (wasb_hybrid is the live
shuttle substrate), parse_trajectory_csv + trajectory_to_action_windows (pure, static, model-
agnostic), candidate_segments persistence (schema v5), play_segments, VideoCompiler stitch with
overlap-merge, rally_gate.py (Gate A net-crossings ? PURE, unit-tested), fusion_features.py
(PURE person-feature math, unit-tested), FusionSegmenter (registered fusion_hybrid, default-OFF,
two identity hooks; net_crossings always persisted, person features + served Gate A/B flag-gated
default-0/OFF), eval harness (metrics/calibration/distill_local/rally_seq_proto), the
skip_ai_handoff fully-local mode. The eval drivers ALREADY consume the trajectory CSV directly
and replay windowing/gating/scoring with no video ? i.e. the video-free seam is de-facto
operational in eval/ (calibrate_wasb.make_predict_fn parses the trajectory once and closes over
trajectory_to_action_windows; eval_partial_labels, export_wasb_segments, calibrate_local all do
this). PLANNED/owner-gated: per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md, NOT started ?
trajectory dir still hardcoded 'output/<family>'), P3 learned fusion (person features into
classifier, gated on golden labels), P4 audio/pose cues, served Gate-A-in-production promotion
(thresholds default 0). NO formal frozen-fixture regression harness keyed on the trajectory CSV
exists as a tracked test yet ? the seam is exploited ad-hoc by eval drivers but not pinned as a
video-free regression suite.",
  "constraints": [
    "Trajectory CSV columns are FIXED: `Frame,Visibility,X,Y` header + order; X,Y are
ORIGINAL-frame pixels (affine-resized back from the 512x288 detector input). A frozen fixture
must capture exactly this.",
    "Windowing needs fps + frame_width alongside the CSV (it normalizes velocity by
frame_width and converts frame->sec by fps). _probe_fps_width reads these from the video via
cv2; for a video-free replay they MUST be captured/pinned (fallbacks 30.0/1280.0 are silent and
would skew results). This is the one non-CSV input the windowing brain needs.",
    "Person features need the per-frame detection dict {frame_idx -> [(track_id,bbox)]} +
frame_width/height; frozen separately from the CSV. Their frame indices must align 1:1 with the
trajectory .frame.",
    "Candidate persistence is detector-agnostic by design (stats/detection_params JSON) ? a
frozen candidate_segments row is a valid second seam but is one transform DOWNSTREAM of the CSV
(windowing already applied), so it cannot regress the windowing brain itself.",
    "gemini and motion segmenters have NO candidate/trajectory stage (final-only) ? they
cannot use the trajectory seam; their regression must freeze provider responses or accept they
touch video.",
    "eval/metrics.match_intervals is GREEDY (not Hungarian) and deterministic; empty matches
yield 0.0 not NaN ? stable for golden assertions.",
    "License guardrail: any frozen-fixture tooling must stay MIT/Apache/BSD; WASB weights are
academic-data-trained (confirm terms before commercial deploy).",
  ],
  "relevantPaths": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",

```

```

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\wasb_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\rally_gate.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\fusion_features.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\person_detector.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\fusion_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\stitcher\\compiler.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_wasb.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\harness.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\features\\rally_seq.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOW_RALLY_DETECTION_WORKS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\MULTI_SIGNAL_FUSION_PLAN.md"
],
"risksGaps": [
    "SOURCES OF NON-DETERMINISM downstream of the trajectory-CSV seam ? these must be controlled or excluded from frozen assertions: (1) FLOAT ? velocity = (dist/frame_width)*(fps/dframes), median/spread in rally_gate, normalized person motion: all float ops, platform-stable but assert with tolerance not exact equality; (2) ORDERING/THREADING ? P3 uses ThreadPoolExecutor(max_workers=ai_max_workers); ai_segments.extend(future.result()) iterates futures in submission order so it IS deterministic, but the underlying provider calls are not (network) ? freeze at the windowing/candidate seam BEFORE P3, or stub the provider; (3) TIMESTAMPS ? candidate_segments/play_segments rows carry created_at DEFAULT CURRENT_TIMESTAMP and the DB autoincrement id ? exclude these columns from any golden comparison; (4) DB id NON-DETERMINISM ? play_segment.id is AUTOINCREMENT, link_candidate_to_segment writes it ? compare on (start_time,end_time,stats) not id; (5) ffmpeg/codecs ? the final reel bytes are NOT reproducible across ffmpeg builds (libx264 version, NVENC vs libx264 fallback, watermark font) ? never assert reel bytes; assert the time_pairs + merge output instead.",
    "fps/frame_width are probed from the VIDEO (cv2) inside the segmenter, not stored in the CSV ? a video-free replay that forgets to pin them silently falls back to 30.0/1280.0 and produces different windows. The frozen fixture MUST include the (fps, frame_width) used.",
    "No tracked frozen-fixture regression test currently pins the trajectory-CSV -> windows -> candidates -> IoU chain end-to-end as a video-free suite; the seam is only exercised ad-hoc by eval CLIs. This is the gap a regression harness would fill.",
    "trajectory_to_action_windows treats visible==False as a hard track break (occlusion gap resets prev) and silently clamps fps<=0->30 / width<=0->1280 ? a fixture with degenerate values would mask real regressions.",
    "Two segmenters (gemini, motion) bypass the seam entirely (no candidate stage); motion additionally fabricates RANDOM metadata/events (random.sample/uniform) so it is inherently non-deterministic and unsuitable for golden assertions.",
    "The person-feature path (fusion, default-OFF) needs a SEPARATELY frozen per-frame detection dict; cv2 frame-seek accuracy varies by codec (±a few frames on real footage), so the captured dict must come from the same decode the fixture pins."
]
},

```

```

{
  "area": "Test suite execution model, committed fixture data, mocking conventions, and the
right home for new auto-discovered per-video regression fixtures",
  "summary": "Tests are pure-pytest auto-discovery (no test runner framework wrapping):
`pytest.ini` sets `testpaths = tests`, `run_all_tests.py` is just a thin `subprocess` wrapper
around `python -m pytest` (defaults to `-q`, passes all args through, prints a PASS/FAIL banner,
returns pytest's exit code). There is NO conftest.py and NO committed CSV/JSON test fixtures
under `tests/` or `backend/eval/` ? every test synthesizes its fixtures inline into pytest's
`tmp_path` (CSVs via `csv.writer`/`p.write_text`, manifests + golden-feature JSON via
`json.dump`). The only committed data files are 3 small JSONs in `eval_baselines/` (the no-
regression baselines/leaderboard, tracked deliberately) plus `config.json` and
`artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json`. The real per-video golden corpus
(trajjectory CSVs, *.rallies.csv labels, *.golden_features.json, human_lovo_manifest.json) lives
ONLY in the gitignored `output/` dir on the owner's box ? it is NOT committed, so any test
depending on it (test_served_gate_a_regression.py) skips cleanly when absent. There IS a clean
load?transform?assert pattern to follow (fusion_compare.load_videos reads
*_golden_features.json), and an existing manifest-driven, skip-if-absent per-video regression
test to mirror (test_served_gate_a_regression.py). New committed, auto-discovered per-video
fixtures + a parametrized test should slot into a NEW fixtures dir UNDER tests/ (e.g.
tests/fixtures/golden/) with the parametrized test in tests/test_*.py globbing that dir.",
  "keyFindings": [
    "DISCOVERY/RUN: pytest.ini (repo root) is the only pytest config ? a single line
`testpaths = tests`. No pyproject [tool.pytest], no markers registered, no conftest.py anywhere
in the repo. run_all_tests.py (repo root, 60 lines) defaults to `-q`, passes every CLI arg
straight through to `python -m pytest`, and propagates pytest's exit code (CI/pre-PR friendly).
The full-suite lane enforces a 70% coverage floor (CODE_MAP S0); run with `python
run_all_tests.py --cov=backend --cov-report=term-missing`.",
    "SKIP/OPTIONAL-DEP CONVENTIONS: two idioms in use ? (a) `pytest.importorskip('cv2')` /
`importorskip('obsreport')` at module top to skip whole modules when a heavy/optional dep is
absent (CI lacks GPU/torch/cv2); (b) a module-level `_skip = pytest.mark.skipif(not _SPECS,
reason=...)` decorator gated on whether real golden artifacts exist on disk. These are how real-
video/GPU tests degrade to SKIP rather than ERROR in CI.",
    "NO COMMITTED TEST FIXTURES: `Glob tests/**/*.csv`, `tests/**/*.json`,
`backend/eval/**/*.csv|json` all return NOTHING. Every fixture is built inline into `tmp_path`.
CSV GT is written with `csv.writer` or `p.write_text('start,end\\n0,10\\n...')`
(test_gt_loader.py, test_eval_regression.py). Per-video golden-feature JSON is written with
`json.dump({name, fps, frame_width, candidates:[{start,end,shuttle},person{},label]},
gts:[[a,b]], ...)` (test_fusion_compare.py::write_video). Manifests are `json.dump([name,
trajectory, fps, frame_width, labels], ...)` (test_fusion_golden_telemetry.py::write_manifest).",
    "COMMITTED DATA (the only tracked non-code data): eval_baselines/heuristic_lovo_n6.json
(689 B, 15 lines ? the LOVO floor + per-video f1 map),
eval_baselines/experiment_leaderboard.json (4.7 KB, 139 lines ? A/B variant rows with
honest_delta_f1 CIs), eval_baselines/gemini_wrapper_2026-06-10.json (1.98 KB). Plus config.json
(2.7 KB) and artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json (untracked here, ADR-008:
manifest committed / weights gitignored). eval_baselines/ is deliberately tracked so a clean
checkout still has the no-regression baselines.",
    "MOCKING ? DETECTOR: injected, not patched. extract_video(spec, detector=None,...) takes
an optional `detector` with a `detections_over_windows(video, windows, frame_skip, progress_cb)`
duck-typed interface; tests pass a `_StubDetector` returning canned `{frames, fps, frame_width,
frame_height}` dicts (test_fusion_compare.py). fusion_golden.run() is tested by monkeypatching
`fusion_golden.extract_video` wholesale.",
    "MOCKING ? WSL: patch the single seam method `_wsl_bash` on a WasbRunner instance with
`unittest.mock.patch.object(r, '_wsl_bash', side_effect=[_proc(rc,stdout,stderr), ...])`, where
`_proc` is a `types.SimpleNamespace(returncode, stdout, stderr)` stand-in for a
CompletedProcess. Higher-level seams `_stage_video`, `_stage_infer_script`, `_wsl_rm_rf`,
`run_predict` are also patched. No real `wsl.exe` is ever invoked (test_wasb_runner.py).",
    "MOCKING ? VIDEO/GPU: no real MP4 decode in the suite. fusion_golden derives the MP4 path
from the labels filename (X.rallies.csv -> X.MP4) but the stubbed detector never opens it.
TrackNetRunner.parse_trajectory_csv (Frame,Visibility,X,Y DictReader) is monkeypatched to return
canned TrajectoryPoint lists, OR fed a tiny real CSV string. RunTelemetry (nvidia-smi/psutil) is
made hermetic by monkeypatching gpu_name + injecting deterministic gpu_fn/sys_fn/clock
collectors (test_fusion_golden_telemetry.py).",
    "EXISTING load?transform?assert PATTERN TO FOLLOW:
backend/eval/fusion_compare.py::load_videos(features_dir) globs `*_golden_features.json`, loads

```

each into experiment.VideoCandidates, and compare() runs honest LOVO; tests assert on the resulting ?F1/CI/sign-test (test_fusion_compare.py). This is the canonical 'load artifact -> run transform -> assert' shape, but today the artifacts are written to tmp_path inside the test, never committed.",

"EXISTING PER-VIDEO REGRESSION TEST TO MIRROR: tests/test_served_gate_a_regression.py is a manifest-driven, per-video, skip-if-absent regression litmus. It resolves MANIFEST from ``<repo_root>/output/human_lovo_manifest.json`` (env-overridable via SERVED_GATE_A_MANIFEST), ``_golden_specs()`` keeps only specs whose trajectory+labels files exist, and ``_skip = skipif(not _SPECS, ...)`` skips the whole module in CI where the gitignored corpus is absent. It loops over each golden video and asserts served-path F1/over-seg invariants. This is the closest template for new per-video regression fixtures ? the gap a committed-fixture approach would close is that it currently CANNOT run in CI (corpus uncommitted).",

"GT LOADER CONTRACT (load-bearing for fixtures):
backend/eval/gt_loader.py::load_gt_intervals reads BOTH golden-CSV conventions ? header ``start,end`` (collector curate export) OR ``start_time,end_time`` (rally-annotator .rallies.csv, extra cols ignored), else positional cols 0,1; timestamps accept decimal seconds or MM:SS/HH:MM:SS; strict=True raises with file:line, strict=False skips+logs. Any committed CSV fixture must conform to one of these two shapes."

```
],
"dataStructures": [
    {
        "name": "Per-video golden-feature JSON (*_golden_features.json)",
        "where": "Written by backend/eval/fusion_golden.py::extract_video/run into gitignored output/; consumed by backend/eval/fusion_compare.py::load_videos; synthesized inline in tests/test_fusion_compare.py::write_video",
        "shapeOrFormat": "{\n  \"name\": str,\n  \"fps\": float,\n  \"frame_width\": float,\n  \"candidates\": [\n    {\n      \"start\": float,\n      \"end\": float,\n      \"shuttle\": {5 SHUTTLE_FEATURE_NAMES: float},\n      \"person\": {5 PERSON_FEATURE_NAMES: float},\n      \"label\": int(0|1)},\n    \"gts\": [[start,end], ...]]. Tiny (a handful of candidates per video in synthetic tests). This is the natural unit for a committed per-video regression fixture."
    },
    {
        "name": "LOVO manifest (human_lovo_manifest.json)",
        "where": "Lives only in gitignored output/; consumed by fusion_golden.run, served_gate_a, calibrate_local; synthesized inline in tests/test_fusion_golden_telemetry.py::write_manifest",
        "shapeOrFormat": "JSON array of specs: [\n  {\n    \"name\": str,\n    \"trajectory\": \"<name>.csv\",\n    \"fps\": float,\n    \"frame_width\": float,\n    \"labels\": \"<name>.rallies.csv\", optional\n    \"video_path\": str}]"
    },
    {
        "name": "Trajectory CSV (TrackNet/WASB predict output)",
        "where": "backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv (DictReader); referenced by manifest 'trajectory' field; only exists in gitignored output/",
        "shapeOrFormat": "CSV header `Frame,Visibility,X,Y`; one row per frame; Visibility==1 means shuttle visible. Parsed into TrajectoryPoint(frame:int, visible:bool, x:float, y:float)."
    },
    {
        "name": "Golden GT / rally-label CSV (two conventions)",
        "where": "backend/eval/gt_loader.py::load_gt_intervals (canonical), annotations.load_annotations, calibrate_wasb.load_golden_gts; synthesized inline in test_gt_loader.py / test_eval_regression.py",
        "shapeOrFormat": "Either header `start,end` (collector export) OR `rally_number,start_time,end_time,ending_reason,sport` (rally-annotator .rallies.csv), OR headerless positional cols 0,1. Timestamps: decimal seconds or MM:SS/HH:MM:SS. -> List[(start:float, end:float)]"
    },
    {
        "name": "Eval baseline / leaderboard JSON",
        "where": "Committed in eval_baselines/; written by backend/eval/experiment.py::record_experiment + regression.save_baseline; read by run_regression.py no-regression gate",
        "shapeOrFormat": "heuristic_lovo_n6.json: {name, description, computed, mean_f1, std, per_video:{vid:f1}}. experiment_leaderboard.json: array of {timestamp, iou, label_set_hash, num_videos, variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta,
```



```

honest_delta_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test}}"
    },
    {
        "name": "WSL CompletedProcess stand-in (_proc)",
        "where": "tests/test_wasb_runner.py",
        "shapeOrFormat": "types.SimpleNamespace(returncode:int, stdout:str, stderr:str) ? fed
via patch.object(runner, '_wsl_bash', side_effect=[...]) to mock every WSL bash call"
    }
],
    "videoDependency": "No test in the suite decodes a real video. The pipeline cannot run here
(dev container lacks GPU/torch/cv2/numpy per CLAUDE.md). Real-video/GPU paths are handled three
ways: (1) inject a stub detector with a duck-typed detections_over_windows() so no MP4 is
opened; (2) monkeypatch TrackNetRunner.parse_trajectory_csv / fusion_golden.extract_video to
return canned data; (3) for tests that genuinely need the real golden corpus (trajectory CSVs +
.rallies.csv labels + MP4s), gate the whole module behind pytest.importorskip('cv2') and/or a
skipif on whether the gitignored output/human_lovo_manifest.json and its referenced files exist
on disk ? these SKIP in CI and only run on the owner's box. The real 6-video golden corpus
(adarsh_avi_singles, kushagra_singles, targettest_doubles, gbaaddy,
testlarge_short/targettest_short, mahadevpura_singles) is NOT committed; only their derived
*_golden_features.json sit in gitignored output/.",
    "builtVsPlanned": "BUILT: full pytest auto-discovery suite (~85 test_*.py files), the
run_all_tests.py wrapper, the load?transform?assert pattern (fusion_compare.load_videos), the
inline-tmp_path fixture idiom, the skip-if-corpus-absent per-video regression litmus
(test_served_gate_a_regression.py), and the committed eval_baselines/ no-regression baselines.
NOT BUILT / GAP the task targets: there are currently NO committed per-video regression fixtures
and NO parametrized test that auto-discovers committed per-video artifacts ? the existing per-
video regression test depends on the uncommitted gitignored output/ corpus, so it never runs in
CI. A new tests/fixtures/golden/ dir of small committed *_golden_features.json + a parametrized
test would make per-video regression CI-runnable for the first time. (No conftest.py exists yet
either, so shared fixture plumbing would be a new addition.)",
    "constraints": [
        "Commercial-clean licensing + privacy: committed fixtures must EXCLUDE minors from any
training/eval pool and must not embed paid-LLM (Gemini) outputs as a training data source
(CLAUDE.md guardrails). Real-player footage frames cannot be committed; synthetic or trajectory-
only/feature-only JSON is the safe form.",
        "Fixtures must be TINY and offline: the suite is fully mocked, no GPU/WSL/network, and CI
has a 70% coverage floor. Committed *_golden_features.json should be small (a handful of
candidates), like the synthetic ones in test_fusion_compare.py ? not the full real corpus.",
        "CSV fixtures MUST conform to one of the two gt_loader conventions (header start,end |
start_time,end_time, or positional) or they will be rejected/skipped by load_gt_intervals
(strict mode raises with file:line).",
        "File-placement rule (CODE_MAP §0): tests go in tests/test_*.py; eval baselines go in
tracked eval_baselines/; pipeline run outputs go in gitignored output/. There is no existing
tests/fixtures or tests/data dir ? adding one is a new convention but consistent with 'a test ->
tests/'. Do NOT put fixtures in output/ (gitignored, won't survive checkout/CI).",
        "Branch from origin/master, stage explicit paths (no git add -A), re-verify HEAD before
commit ? shared checkout / concurrent sessions (CLAUDE.md + memory gotcha-shared-checkout-
branch-collisions).",
        "MANIFEST.in / hatchling packaging: pyproject lists packages=[backend, training]; a
tests/fixtures dir is fine for pytest (run from source tree) but would not be packaged into the
wheel ? acceptable since tests aren't shipped."
    ],
    "relevantPaths": [
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_all_tests.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pytest.ini",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pyproject.toml",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_eval_regression.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_gt_loader.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_fusion_golden_telemetry.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_compare.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_served_gate_a_regression.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-

```

```

indexer\\tests\\test_per_user_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_wasb_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_compare.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\per_user_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval\\baselines\\heuristic_lovo_n6.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval\\baselines\\experiment_leaderboard.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md"
],
"risksGaps": [
  "RECOMMENDED PLACEMENT for new auto-discovered per-video fixtures: create
tests/fixtures/golden/ holding small committed *_golden_features.json (one per synthetic video,
matching the fusion_golden output schema), and a new tests/test_golden_regression.py that does
`FIXTURE_DIR = Path(__file__).parent/'fixtures/'golden` then
`@pytest.mark.parametrize('fixture', sorted(FIXTURE_DIR.glob('*_golden_features.json'))),
ids=lambda p: p.stem)`. This mirrors fusion_compare.load_videos (glob+json.load) but reads
COMMITTED files instead of tmp_path, and mirrors test_served_gate_a_regression.py's per-video
loop ? except it runs in CI because the fixtures are committed. Resolve paths via
Path(__file__).parent, NEVER cwd (agent cwd resets between bash calls; tests may run from any
dir).",
  "There is no conftest.py today. If the parametrized test needs a shared loader/fixture,
add tests/conftest.py (new file) with a small `load_golden_fixture` helper or a `golden_videos`
fixture ? but a module-local glob is simpler and matches the existing zero-conftest convention;
prefer that unless multiple test files need it.",
  "Do NOT reuse output/ for committed fixtures ? it is gitignored and won't survive a clean
checkout or exist in CI (that is exactly why test_served_gate_a_regression.py skips in CI). The
fixtures must be inside tests/ to be tracked and discovered.",
  "Privacy/licensing risk if real footage-derived data is committed: the real 6-video corpus
is owner footage; commit only SYNTHETIC or fully-anonymized feature JSON (no frames, no minors),
consistent with the inline-synthetic data already used in test_fusion_compare.py. If anchoring
on real numbers, store only the derived F1/baseline in eval_baselines/ (already the convention),
not raw player data.",
  "A committed-fixture regression test should assert against a committed expected value
(e.g. an eval_baselines/ entry or an inline EXPECTED constant) using the existing
regression.regress / experiment.compare / metrics.score transforms ? reuse those, do not write
new scoring math (CODE_MAP warns calibrate_wasb.load_golden_gts + experiment harness are widely-
shared APIs).",
  "Marker hygiene: no custom pytest markers are registered (no [tool.pytest] markers
config). If the new test introduces a marker (e.g. @pytest.mark.golden), register it in
pytest.ini's `markers =` or it will emit PytestUnknownMarkWarning; simpler to rely on
parametrize + skipif like the rest of the suite."
]
},
{
  "area": "Repo constraints for committing structured regression fixtures (file-placement,
gitignore, git-lfs, schema/version conventions, licensing/privacy guardrails, size budget)",
  "summary": "The repo deliberately commits NO raw/derived video data today: there are zero
tracked .csv/.jsonl/fixture files, tests/ has no data dirs (tests build synthetic data in-code),
and the ONLY committed structured data is tiny aggregate-metric JSON in eval_baselines/ (e.g.
heuristic_lovo_n6.json ? mean_f1 + per-video scores, no per-frame data). The file-placement
table (CODE_MAP $0) routes pipeline outputs (trajectories, features, DBs, frames) to gitignored
output/, and a dedicated doc ? docs/GOLDEN_DATA_SHARING.md (#175) ? explicitly states the golden
corpus is PROPRIETARY and its derived features must live in a PRIVATE OneDrive folder OUTSIDE
the repo, never committed, with a guardrail forbidding symlinking it into a tracked path. git-
lfs is INSTALLED (3.5.1, global filters) but NOT in use (no .gitattributes anywhere). On the
licensing/privacy question: committing detector-derived numeric streams (shuttle X/Y, segment

```

intervals, person/audio features) is defensible because the golden videos are owned/licensed by the founder (self-recorded + ShuttleSet, never end-user uploads) and the streams are abstract numbers, not frames ? BUT the repo's OWN convention currently says keep them out of git (private OneDrive), and minors must be excluded, so any decision to commit even derived data is a product/privacy call that is owner-gated, not a free engineering choice.",

"keyFindings": [

"(1) WHERE FIXTURES BELONG: Per docs/CODE_MAP.md \$0 file-placement table ? tests go in tests/test_*.py (pytest auto-discover, 70% coverage floor, must be pure/offline/mocked, no GPU/WSL/network). Small TRACKED metric baselines go in eval_baselines/ ('Tracked ? survives a clean checkout; the no-regression gate reads it'). Model-artifact MANIFESTS go in artifacts/<name>/<ver>/ (manifest committed, weights gitignored ? ADR-008). Pipeline run outputs (reels, TRAJECTORIES, DBs, frames) go in output/ = GITIGNORED.",

"(2) WHAT IS GITIGNORED: .gitignore ignores output/, output_dir/, scratch/, .env/, all *.db/*.db-journal/*.db-wal, *.pt (weights), .env*, coverage, __pycache__, and specific backend/static highlight mp4s. NOTE: artifacts/ itself is NOT in .gitignore (it currently shows as untracked in git status) ? only model WEIGHTS inside it are meant to be gitignored per ADR-008, the manifest is committed. scratch/ is fully gitignored+excluded from ruff/pytest.",

"(3) GIT-LFS: git-lfs v3.5.1 is installed and its filters are registered globally (filter.lfs.clean/smudge/process, required=true), BUT there is NO .gitattributes file anywhere in the repo ? so LFS tracks nothing and is effectively NOT in use. LFS is AVAILABLE if needed but the repo has chosen the 'keep big data out of git' path (output/ gitignored + private OneDrive sync) instead of LFS.",

"(4) NO EXISTING DATA FIXTURE PRECEDENT: `git ls-files` finds ZERO committed .csv/.jsonl files and no tests/ data/fixture/golden directories. Tests (e.g. test_fusion_golden_telemetry.py) build manifests + synthetic data inline in code with monkeypatch/tmp_path. The ONLY committed structured-data precedent is eval_baselines/*.json ? and those are tiny AGGREGATE metrics (mean_f1, std, per_video score dict), NOT raw per-frame or per-window derived streams.",

"(5) PROPRIETARY-DATA GUARDRAIL IS EXPLICIT: docs/GOLDEN_DATA_SHARING.md (status 2026-06-15, #175) says the GPU-extracted golden artifacts (trajectory CSVs, *_golden_features.json, manifests) sync via a PRIVATE OneDrive folder at %USERPROFILE%\OneDrive\rally-annotator\golden-data\ ? OUTSIDE the repo ? precisely so large/proprietary data is 'never committed to git.' Guardrail: 'features derive from the Proprietary corpus ? keep the OneDrive folder private (no public share links)' and 'do not symlink the shared folder into a tracked path.'",

"(6) SCHEMA/VERSION CONVENTIONS TO MIRROR: DB uses a PRAGMA user_version migration ladder in backend/infrastructure/database.py (currently at v5; ordered `if version < N:` blocks, additive ALTERs that preserve prior rows, tests assert the expected version so drift fails CI). Detector data is DETECTOR-AGNOSTIC: common fields are columns, detector-specific metrics go in a `stats` JSON blob keyed by a `detector` string ? a new detector reuses the table. eval_baselines JSON carries provenance fields (name, description, `computed` repro command, metric values). Golden features JSON has a stable shape: {name, fps, frame_width, candidates:[{start,end,shuttle{5 feats},person{5 feats},label}], gts:[a,b]}. Any fixture should likewise carry a version/schema tag + provenance and follow the column-vs-stats-JSON split.",

"(7) VIDEO PROVENANCE IS CLEAN-BY-DESIGN: docs/GOLDEN_SET_VENDORING.md ? 'Golden sets are built only from footage we license or own ? starting with the founder's personal collection, plus ShuttleSet, plus purpose-recorded clips. No end-user uploads ever flow to a vendor.' So the source is owned/self-recorded + ShuttleSet (broadcast contrast only), NOT third-party broadcast scraping.",

"(8) MINORS / PRIVACY GUARDRAIL: CLAUDE.md + COMMERCIALIZATION.md C14 + OWNED_MODEL_IMPLEMENTATION_PLAN mandate 'exclude minors' and 'per-user training data needs a separate explicit opt-in.' FIELD_VISIT_PLAYBOOK: 'sessions with minors are not recorded for our library ? adult sessions only.' KHELSUTRA_PRODUCT_OVERVIEW: 'Footage involving minors is never used to improve the tool and is never published.' Any committed fixture must be guaranteed minor-free at source.",

"(9) LICENSING GUARDRAIL (for completeness): CLAUDE.md ? paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving ONLY, NEVER a training data source; all code+data+weights must be MIT/Apache-2.0/BSD. Derived numeric streams from owned video do not implicate paid-LLM-output rules (they come from MIT-licensed WASB/TrackNet detectors, not from Gemini)."

],

"dataStructures": [

{

"name": "Trajectory CSV (per-video shuttle stream)",

```

    "where": "Produced to gitignored output/ (predict writes <stem>_predict.csv); parsed by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` ? exactly one row per video frame
(int frame index, 0/1 visibility, pixel X, pixel Y). Pure numeric, no imagery."
},
{
    "name": "*_golden_features.json (per-video derived feature record)",
    "where": "backend/eval/fusion_golden.py::extract_video return value; written one JSON
per video to out_dir (output/ or the private OneDrive features/ folder)",
    "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end, shuttle:{~5
fps/resolution-invariant feats}, person:{~5 person-cue feats}, label:int}],
gts:[{start,end},...]. The replayable CPU re-scoring substrate; small (a few KB?tens of KB)."}
},
{
    "name": "eval_baselines/*.json (the ONLY committed structured-data precedent)",
    "where": "eval_baselines/heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json,
experiment_leaderboard.json ? TRACKED in git",
    "shapeOrFormat": "Tiny aggregate-metric JSON: {name, description, computed:<repro
command>, mean_f1, std, per_video:{video_name: f1, ...}}. Carries provenance; NO per-frame/per-
window data."
},
{
    "name": "candidate_segments table (detector-agnostic DB schema)",
    "where": "backend/infrastructure/database.py, migration v1 (PRAGMA user_version=1)",
    "shapeOrFormat": "Columns: id, video_id, detector(str), chunk_index, start_time,
end_time, duration, confidence, stats(JSON for detector-specific metrics),
detection_params(JSON), confirmed_segment_id, human_label, notes, user_id(v4, nullable). Common
fields=columns, detector-specific=stats JSON ? the reuse pattern a fixture should mirror."
},
{
    "name": "PRAGMA user_version migration ladder",
    "where": "backend/infrastructure/database.py::_init_db",
    "shapeOrFormat": "Ordered `if version < N:` blocks; currently at v5 (v1
candidate_segments, v2 jobs, v3 self-scheduling, v4 user_id columns, v5 user_models). Additive
ALTERs preserve rows; tests assert expected version (drift fails CI)."}
}
],
"videoDependency": "High but CLEAN. The derived streams (shuttle X/Y trajectories,
candidate-window features, person/audio cues) are extracted FROM the golden videos, so fixtures
encode information about proprietary footage. However: (a) the videos are owned/licensed by the
founder (self-recorded + ShuttleSet), never third-party broadcast scraping and never end-user
uploads (GOLDEN_SET_VENDORIZING.md); (b) the fixtures are abstract NUMERIC streams (frame index,
pixel coords, scalar features) ? NOT frames/imagery, so they carry no biometric/identifiable
content; (c) the detectors that produce them (WASB/TrackNet) are MIT-licensed and the streams
contain no paid-LLM output, so the licensing guardrail is not implicated. The binding constraint
is the repo's OWN convention (GOLDEN_DATA_SHARING.md) that these proprietary-derived artifacts
live in private OneDrive and are NOT committed ? committing them is a product/privacy reversal
that is owner-gated.",
"builtVsPlanned": "BUILT: the gitignore rules; eval_baselines/ as the tracked-baseline
pattern; the PRAGMA user_version ladder (v5) with detector-agnostic stats-JSON tables;
backend/eval/fusion_golden.py producing the *_golden_features.json streams; the trajectory CSV
parse path; the test suite's synthetic-data-in-code pattern (no committed data fixtures).
CONVENTION/PLAN (not code-enforced): GOLDEN_DATA_SHARING.md private-OneDrive sync (#175,
status='convention + plan 2026-06-15' ? the optional scripts/sync_golden.py and RALLY_GOLDEN_DIR
env var are NOT required/built yet); the hard train/eval-leakage guardrails
(permissions/approval at ingest+export) are explicitly 'TODO ? design with the user' in
GOLDEN_SET_PHASE1_VIDEOS §5 / BACKLOG; the consent ledger (contribution_consent, minors-
exclusion at the writer) is PERSONALIZATION_PLAN v6 'pending.' git-lfs is available but unused
(no .gitattributes).",
"constraints": [
    "Committed fixtures = tests/test_*.py (pure/offline/mocked, 70% cov floor) for logic, and
eval_baselines/*.json for tracked metric baselines. Bulk derived data does NOT go in git ?
output/ is gitignored and the golden artifacts are designed to live in private OneDrive, NOT the
repo.",
    "Do NOT commit raw frames/video, *.db, *.pt weights, or large per-frame streams ? all

```

gitignored by convention. Model artifacts: commit the manifest in artifacts/<name>/<ver>/,
gitignore the weights (ADR-008).",

"Mirror the detector-agnostic schema: common fields as columns/keys + a `stats`/params
JSON blob keyed by a `detector` string; carry a version/schema tag + provenance (like
eval_baselines' `computed` repro command and like PRAGMA user_version).",

"Licensing: every dependency (code+data+weights) MIT/Apache-2.0/BSD only; paid LLMs are
serving-only and NEVER a training/data source ? derived streams must come from the MIT
WASB/TrackNet detectors, not Gemini output.",

"Privacy: exclude minors at the source; per-user/contributed data needs explicit separate
opt-in + consent ledger; footage involving minors is never used to improve the tool. Committing
derived data from such footage would violate this even though it's 'just numbers.'",

"Branching safety (CLAUDE.md): branch from origin/master explicitly, re-verify HEAD before
commit, stage explicit paths (never git add -A) ? relevant when actually landing any fixture PR.
NOTE: `git pull --ff-only` failed at session start ('Cannot fast-forward to multiple branches')
? the local checkout may lag origin; treat as possibly stale."

```
],  
"relevantPaths": [  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\heuristic_lovo_n6.json",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\COMMERCIALIZATION.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\CLAUDE.md",  
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_golden_telemetry.py"  
],  
"risksGaps": [  
  "RECOMMENDED DESIGN TO STAY CLEAN: commit only (a) tiny aggregate/metric fixtures and  
  small DERIVED NUMERIC streams (trajectory CSV rows, feature JSON) ? never frames/imagery; (b)  
  from videos guaranteed owned/licensed AND minor-free at source; (c) with an explicit  
  provenance/license/consent tag + a version/schema tag in each fixture. This keeps licensing  
  clean (MIT detectors, no paid-LLM output) and privacy clean (no biometrics, no minors).",  
  "POLICY CONFLICT TO RESOLVE WITH OWNER: GOLDEN_DATA_SHARING.md currently says the  
  proprietary-derived golden features stay in PRIVATE OneDrive and are NOT committed. A 'commit a  
  small regression fixture to git' design REVERSES that convention. Because the corpus is labeled  
  Proprietary, this is a product/privacy decision (owner-gated), not a unilateral engineering  
  choice ? get explicit sign-off and ideally commit a redacted/synthetic or a tiny owner-cleared  
  subset rather than the full corpus.",  
  "SIZE BUDGET: corpus is n=6 owned golden videos today  
  (eval_baselines/heuristic_lovo_n6.json + LOVO docs). Per-video trajectory CSV = one row per  
  frame of `Frame,Visibility,X,Y` (~20-25 bytes/row); a ~2-4 min clip at 30-60fps ? 3.6k-14.4k  
  rows ? ~80KB-350KB per video ? full n=6 trajectory set ? <2MB. The *_golden_features.json (one  
  record per candidate window, ~10 feats) is far smaller ? a few KB to tens of KB per video. So a  
  numeric-only regression fixture for all 6 videos is comfortably git-sized (low single-digit MB);  
  git-lfs/.gitattributes is NOT needed at this scale.",  
  "GAP: no code-enforced guardrail yet prevents derived data from a minors/unconsented or  
  unlicensed video being committed ? the leakage/consent guards are TODO (PHASE1_VIDEOS $5,  
  PERSONALIZATION_PLAN v6 pending). Until those exist, the minor-free/owned guarantee rests on  
  manual discipline; a committed fixture should carry an explicit per-video provenance+consent  
  assertion.",  
  "GAP: artifacts/ is untracked and NOT listed in .gitignore ? if a fixture-generating PR  
  also touches artifacts/, be careful not to accidentally stage weights; stage explicit paths  
  only.",
```

```
"STALENESS FLAG: `git pull --ff-only` failed ('Cannot fast-forward to multiple branches')
? local master may lag origin/master; re-fetch and branch from origin/master before landing any
fixture PR (per CLAUDE.md shared-checkout safety)."
```

```
    ]
  }
]
```

THREE PROPOSALS (JSON):

```
[
  {
    "approachName": "Freeze the fusion_golden feature substrate as committed eval_baselines
fixtures + a frozen score snapshot, replayed in CI by the existing experiment.compare engine",
    "summary": "The video-free regression engine already exists and runs end-to-end on CPU:
fusion_compare.load_videos() globs per-video *_golden_features.json, builds
experiment.VideoCandidates, and experiment.compare() scores them deterministically (bootstrap is
seeded seed=0; per-video P/R/F1/mIoU are pure interval math via metrics.py). The ONLY missing
piece is committed inputs. So the minimal design commits the *_golden_features.json substrate
(the proven freeze seam: derived numeric features + golden GT intervals, no frames/video) into
the tracked eval_baselines/ tree, freezes the expected per-video scores into one snapshot JSON,
and adds a parametrized pytest that re-derives scores from the committed fixtures and asserts
they match the snapshot within tolerance. A contributor on a fresh clone can then refactor
windowing/gating/scoring/feature code and the suite catches any per-video score drift with zero
video, GPU, WSL, or network. The set grows automatically because the SAME already-built
fusion_golden.py extractor that the owner runs on each new golden video emits exactly the
committed artifact shape ? adding a video is: drop its *_golden_features.json into the fixtures
dir, re-snapshot, commit. Reuses metrics.py / segmentation_metrics.py / experiment.compare /
fusion_compare.load_videos / the eval_baselines/ committed-location convention wholesale; the
only net-new code is the snapshot file, a ~40-line test, and a tiny --snapshot flag on
fusion_compare. PRIVACY/POLICY NOTE: committing per-video derived features reverses the current
GOLDEN_DATA_SHARING.md private-OneDrive convention, so it is owner-gated ? the safe shippable
form is to commit a SYNTHETIC or owner-cleared minor-free subset (the features are
scale/translation-invariant scalars, not imagery), exactly like the synthetic fixtures already
used in test_fusion_compare.py.",
    "buildsOnExisting": [
      "backend/eval/experiment.py::compare / evaluate_variant / VideoCandidates / Variant ? the
pure, deterministic LOVO scoring engine reused verbatim (no new scoring math)",
      "backend/eval/fusion_compare.py::load_videos ? already globs *_golden_features.json and
builds VideoCandidates; the test reuses it pointed at the committed fixtures dir instead of
gitignored output/",
      "backend/eval/metrics.py + segmentation_metrics.py ? pure stdlib interval P/R/F1/IoU;
deterministic, empty-match yields 0.0 not NaN (stable assertions)",
      "backend/eval/eval_stats.py::paired_delta_ci/bootstrap_ci ? deterministic with default
seed=0, so the ?F1 CI + sign test are reproducible and snapshot-able too",
      "backend/eval/fusion_golden.py ? the SAME extractor the owner already runs per golden
video; it emits the exact committed artifact shape, so 'add a video' needs no new producer
(auto-grow comes for free)",
      "eval_baselines/ ? the established TRACKED home that 'survives a clean checkout'
(heuristic_lovo_n6.json, experiment_leaderboard.json); mirror its provenance-carrying JSON
convention",
      "backend/eval/regression.py::gt_hash + experiment.label_set_hash/spec_hash ? the existing
fingerprint contract; the snapshot embeds these so a comparison against changed labels/recipe is
flagged incommensurable, not silently trusted",
      "tests/test_fusion_compare.py ? the load->transform->assert template + the synthetic
*_golden_features.json _write_video() shape to reuse for synthetic fixtures",
      "backend/eval/splits.py::assert_disjoint ? existing pure train/eval guard to wire in so a
fixture can't leak between a trained scorer and the regression baseline"
    ],
    "seams": [
      "PRIMARY (the one to freeze): the per-video *_golden_features.json substrate ? {name, fps,
frame_width, candidates:[{start,end,shuttle{5 feats},person{5 feats},label}], gts:[{s,e}...]}.
This is downstream of the GPU trajectory pass and person-detect pass but UPSTREAM of all pure
transforms (candidate windowing already applied, but feature->score->LOVO is what the test
exercises). It is the de-facto operational video-free seam (fusion_compare already consumes it)
and contains only scale/translation-invariant scalars + GT intervals ? repo-safe where raw
frames/trajectory CSVs are riskier.",
```

"ALTERNATIVE (deeper, optional later): the raw trajectory CSV `Frame,Visibility,X,Y` + pinned (fps, frame_width). Freezing here would also regress the windowing brain (trajectory_to_action_windows) and Gate-A net-crossings, not just feature->score. Heavier (one row per frame, ~80-350KB/video) and re-derives features each run. Recommend deferring to a second PR; the feature-JSON seam is the smaller delta that still catches scoring/feature-consumption refactors.",

"NOT a seam: gemini/motion segmenters (no candidate stage; motion fabricates random metadata) ? explicitly out of scope; the trajectory-hybrid rally-quality track is what gets the fixture."

],

"fixtureLayout": "Create eval_baselines/golden_fixtures/ (TRACKED, sibling to the existing committed baselines ? keeps all 'survives-a-clean-checkout' eval data in one tracked tree and avoids inventing a new tests/fixtures/ convention). Layout:

eval_baselines/golden_fixtures/<corpus-tag>/<video_name>_golden_features.json (one file per golden video; corpus-tag e.g. 2026-06-n6 matching the GOLDEN_DATA_SHARING corpus-tag convention) + eval_baselines/golden_fixtures/<corpus-tag>/MANIFEST.json (repo-relative list of the fixture files + per-video provenance/consent/license tag + the operating-point/windowing the substrate was frozen at). The frozen expected-scores snapshot lives at

eval_baselines/golden_regression_snapshot.json. Rationale for eval_baselines/ over tests/fixtures/: the grounding shows eval_baselines/ is the proven tracked home for committed eval artifacts and is read by the no-regression machinery; tests/ has zero committed data fixtures today, so this stays consistent with the established convention rather than starting a new one. The test resolves the dir via Path(__file__) (repo-root-anchored), never cwd.",

"artifactSchema": "Reuse the EXISTING fusion_golden output schema verbatim (do not invent a parallel format): per-video {name:str, fps:float, frame_width:float, candidates:[{start:float, end:float, shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation}, label:int}}, gts:[[start,end],...]]. Add a top-level \"schema_version\":1 + \"provenance\":{\"video_owned:true, minor_free:true, source:\"owned|synthetic\", license:\"owned|Apache-detector-derived\", windowing:\"proposal|served\", operating_point:{velocity_thresh, merge_gap, min_window}} block so every fixture self-declares its freeze conditions (the Gate-A eval!=serve cautionary tale: a baseline must record WHICH windowing it was measured at). The MANIFEST.json carries label_set_hash (over all gts) + the corpus-tag. The snapshot file: {\"schema_version\":1, \"iou\":0.5, \"seed\":0, \"corpus_tag\":..., \"label_set_hash\":..., \"spec_hash_shuttle_only\":..., \"spec_hash_fusion\":..., \"per_video\":{\"<name>\":{\"shuttle_only\":{\"precision,recall,f1,mean_iou}, \"fusion\":{\"...}}}, \"aggregate\":{\"shuttle_only:{...}, fusion:{...}}, \"paired_delta_f1\":{\"mean_delta,ci_low,ci_high,sign_test:{wins,losses,p_value}}} ? i.e. the deterministic experiment.compare() output, frozen.",

"extractionMechanism": "No new extractor. The owner's already-built `python -m backend.eval.fusion\_golden --manifest <m> --out <dir>` (GPU-free given pre-extracted trajectory CSV + person dict) writes *_golden_features.json today into gitignored output/. The 'extraction step' for the fixture is literally: copy the chosen video's <name>_golden_features.json from output/ (or the private OneDrive features/<corpus-tag>/) into eval_baselines/golden_fixtures/<corpus-tag>/, and (for a synthetic/cleared fixture) optionally regenerate via the test_fusion_compare_write_video() shape. Add ONE small CLI flag to fusion_compare.py: `--snapshot eval\_baselines/golden\_regression\_snapshot.json` that loads the committed fixtures dir, runs compare(), and writes the deterministic per-video+aggregate scores + hashes to the snapshot file (this is just experiment.compare() output serialized ? ~15 lines reusing existing functions). That single command both freezes the baseline initially and re-freezes it when the corpus grows or an intentional scoring change lands.",

"regressionGates": "A new tests/test_golden_regression.py: FIXTURE_DIR = Path(__file__).parent.parent/'eval_baselines'/ 'golden_fixtures'; load the snapshot once; `@pytest.mark.parametrize('name', sorted(fixture video names))` so EACH committed video is its own test case (clear per-video failures). Each case: load that video's fixture via fusion_compare.load_videos (or a single-file loader), run experiment.compare() over the full committed set at the snapshot's iou+seed, and assert the recomputed per-video shuttle_only/fusion P/R/F1/mean_iou == the snapshot value within abs tol 1e-6 (float-stable, not exact-equality, per the metrics determinism note). A separate test asserts the aggregate + paired ?F1 CI + sign test match the snapshot (also deterministic via seed=0). A guard test asserts label_set_hash(committed fixtures) == snapshot.label_set_hash, so a fixture edited without re-snapshotting fails LOUDLY as incommensurable rather than silently drifting (reuses the existing gt_hash/label_set_hash contract). If the fixtures dir is empty the module skips (skipif) exactly like test_served_gate_a_regression.py ? but unlike that test, it RUNS in CI

because the fixtures are committed. This is what catches a contributor's refactor side-effect: any change to windowing-feature-consumption / scoring / fusion recipe that moves a per-video score trips the assert.",

"repoSizeStrategy": "Comfortably git-sized; no git-lfs/.gitattributes needed. Each *_golden_features.json is one record per candidate window (~10 scalar feats + interval), a few KB to low tens of KB per video; n=6 corpus is well under ~200KB total, the snapshot is a single small JSON. (For contrast the raw-trajectory alternative seam would be ~80-350KB/video, still <2MB for n=6 ? also git-sized, but the feature-JSON seam is ~10-50x smaller, another reason to pick it.) The corpus grows only ~tens-of-KB per added video; LFS stays unused as the repo already chose 'keep big data out, commit small derived metrics'. Keep fixtures TINY and synthetic-where-possible to honor the offline/mockd suite norm.",

"licensingHandling": "Detector-derived numeric features come from MIT/Apache WASB/TrackNet detectors, NOT from any paid LLM, so the 'paid-LLM-never-a-training/data-source' guardrail is not implicated. Per-fixture provenance block asserts video_owned:true (founder collection / ShuttleSet / purpose-recorded ? never end-user uploads, never broadcast scraping) and license tag. Commit ONLY derived scale/translation-invariant scalars + interval labels ? never frames, never raw player coordinates beyond the abstracted features, never video. This keeps every committed dependency MIT/Apache/BSD-clean.",

"determinismHandling": "Fully deterministic and verified: per-video P/R/F1/mean_iou are pure interval math (metrics.match_intervals is greedy+deterministic, empty->0.0 not NaN); the LOVO learned-variant training is deterministic; the bootstrap CI + sign test are seeded (eval_stats default seed=0) so the ?F1 CI snapshot is reproducible. Controls: (1) assert with abs tol 1e-6 not exact float equality (platform-stable float ops); (2) PIN iou + seed in the snapshot and pass them explicitly in the test; (3) the features are already fps/resolution-invariant so no fps/frame_width fallback (30.0/1280.0) hazard at this seam ? but the fixture still records fps/frame_width for provenance; (4) exclude any wall-clock/DB-id/created_at fields (none exist in this seam) and never assert on reel bytes/DB ids. If the deeper trajectory-CSV seam is added later, the fixture MUST additionally pin (fps, frame_width) since windowing silently falls back otherwise.",

"versioningStrategy": "schema_version:1 on every fixture + the snapshot (mirrors the PRAGMA user_version additive-ladder discipline). The snapshot is regenerated (not hand-edited) by the --snapshot flag whenever the corpus grows or an INTENTIONAL scoring change lands; the diff in the PR shows exactly which per-video numbers moved and why (an intentional improvement re-snapshots in the same PR; an accidental regression fails CI before merge). label_set_hash + spec_hash_* travel in the snapshot so changed labels or a changed fusion recipe are flagged incommensurable rather than silently re-baselined. corpus-tag (e.g. 2026-06-n6) versions the fixture set so an expanded corpus is a new tag/dir, preserving the old frozen set for reproducibility of prior published numbers (ties the paper-coauthor empirical-backbone aim).",

"ciIntegration": "Zero new CI wiring needed for the core goal: the test is plain pytest, auto-discovered by the existing `testpaths = tests` / run_all_tests.py lane that already runs in .github/workflows/ci.yml with the 70% coverage floor. It runs in the no-GPU/no-cv2/no-network lane because the committed fixtures need none of those. (Optional, second-PR nicety: also add a CI step `python -m backend.eval.fusion\_compare --features-dir eval\_baselines/golden\_fixtures/<tag> --check-snapshot` that exits non-zero on drift, mirroring run_regression.py's exit 0/1/3 contract ? but the pytest assertion already gates merges, so this is redundant unless the owner wants a standalone gate.) No marker registration needed; rely on parametrize + skipif like the rest of the suite to avoid PytestUnknownMarkWarning.",

"contributorWorkflow": "Contributor (any clone): runs `python run\_all\_tests.py` ? test_golden_regression.py loads committed fixtures, recomputes scores, and a refactor that changes windowing/feature-consumption/scoring/fusion-recipe behavior fails the per-video assert with a clear name (e.g. 'kushagra_singles fusion f1 0.157->0.142'). A legitimately score-changing PR re-runs `python -m backend.eval.fusion\_compare --features-dir eval\_baselines/golden\_fixtures/<tag> --snapshot eval\_baselines/golden\_regression\_snapshot.json`, eyeballs the diff, and commits the new snapshot in the same PR (the numbers moving is the review artifact). Owner (adding a golden video): runs fusion_golden on the new video (GPU box), drops its <name>_golden_features.json into eval_baselines/golden_fixtures/<tag>/, updates MANIFEST.json provenance, re-runs --snapshot, commits ? the set has grown and the new video is now a CI-enforced regression case automatically. Branch from origin/master, stage explicit paths (the fixture file + snapshot + test), never git add -A (shared-checkout safety).",

"pros": [

"Smallest possible delta: the scoring engine, loader, stats, fingerprints, and committed-location convention all already exist ? net-new code is ~1 snapshot JSON + a ~40-line parametrized test + a ~15-line --snapshot flag.",

"Achieves the exact goal: a fresh clone runs rally-segmentation regression with ZERO video/GPU/WSL/network in the existing CI lane ? catches refactor side-effects per-video.",


```

    "Lights up coverage that is silently skipped today (test_served_gate_a_regression and
golden-litmus tests skip in CI for lack of committed corpus); committed fixtures make per-video
regression CI-runnable for the first time.",
    "Auto-grows for free: the same fusion_golden extractor the owner already runs emits the
committed shape, so 'add a golden video' is copy-file + re-snapshot, no new producer.",
    "Fully deterministic (seeded bootstrap, pure interval math) so assertions are stable and
the ?F1 CI/sign-test honest-stats are snapshot-able ? keeps the 'numbers inform' empirical
backbone reproducible across clones.",
    "Tiny + git-sized (well under ~200KB for n=6); no git-lfs, no .gitattributes, no infra.",
    "Privacy/licensing-clean form available: scale/translation-invariant scalars + interval
labels only, no frames; synthetic or owner-cleared minor-free subset honors guardrails.",
    "Records the freeze operating-point/windowing in provenance, avoiding the Gate-A
eval!=serve trap of a baseline that mispredicts served behavior."
  ],
  "cons": [
    "Reverses the current GOLDEN_DATA_SHARING.md convention (proprietary-derived features live
in private OneDrive, NOT git) ? committing even derived features is an OWNER-GATED
product/privacy decision, not a free engineering call. Mitigation: ship a synthetic or owner-
cleared minor-free subset first.",
    "The chosen seam is one transform DOWNSTREAM of the trajectory CSV, so it regresses
feature-consumption->score->LOVO but NOT the windowing brain (trajectory_to_action_windows) or
Gate-A net-crossing derivation. Mitigation: add the trajectory-CSV seam in a later PR if
windowing-brain regression coverage is wanted.",
    "An intentional scoring improvement requires a manual re-snapshot in the same PR; forget
it and CI fails (this is by design but is a small friction).",
    "Synthetic fixtures catch logic/refactor regressions but won't reflect real-corpus score
distributions; real owner-cleared fixtures are needed for the numbers to mean what
eval_baselines/heuristic_lovo_n6.json means.",
    "splits.assert_disjoint is not yet wired into this path; if the same fixture ever trains a
pinned scorer AND serves as a regression baseline, leakage is possible until that guard is
connected."
  ],
  "implementationSteps": [
    "PR1 (shippable, owner-gated on the privacy reversal): (a) Create
eval_baselines/golden_fixtures/<corpus-tag>/ and commit a SMALL set of *_golden_features.json ?
start SYNTHETIC (reuse test_fusion_compare _write_video shape) so the PR is mergeable without
the privacy sign-off, or an owner-cleared minor-free real subset if approved.",
    "PR1 (b): Add MANIFEST.json (repo-relative fixture list + per-video
provenance:{video_owned,minor_free,source,license>windowing,operating_point} + corpus-tag +
label_set_hash) and add schema_version+provenance blocks to each fixture.",
    "PR1 (c): Add a --snapshot flag to backend/eval/fusion_compare.py (default-OFF) that loads
--features-dir, runs the existing compare(), and serializes the deterministic per-video +
aggregate scores + iou + seed + label_set_hash + spec_hashes to
eval_baselines/golden_regression_snapshot.json. Run it once to generate the committed
snapshot.",
    "PR1 (d): Add tests/test_golden_regression.py ? Path(__file__)-anchored fixtures dir, load
snapshot, @parametrize per video, recompute via experiment.compare at pinned iou+seed, assert
per-video + aggregate + paired-?F1 match within 1e-6, plus a label_set_hash guard test; module
skipif fixtures dir empty.",
    "PR1 (e): Run `python run_all_tests.py --cov=backend` locally to confirm green + coverage
holds; branch from origin/master, stage explicit paths (the fixtures, snapshot, test,
fusion_compare.py), re-verify HEAD before commit.",
    "PR1 (f): Document the add-a-golden-video workflow (copy *_golden_features.json into the
fixtures dir + re-run --snapshot + commit) in docs/EVALUATION.md or GOLDEN_DATA_SHARING.md, and
note the convention reversal/decision.",
    "PR2 (optional, later): wire splits.assert_disjoint into the path if a pinned scorer is
ever trained on these fixtures; and/or add the deeper raw-trajectory-CSV seam (pinning
fps+frame_width) so the windowing brain + Gate-A net-crossings are also video-free-regressed;
and/or add a standalone fusion_compare --check-snapshot CI step with run_regression-style exit
codes."
  ]
},
{
  "approachName": "Manifest-indexed frozen-seam fixtures + dual regression gates
(characterization snapshot + golden quality-floor), reusing the existing pure-CPU eval stack",

```

"summary": "Add a committed, version-tagged per-video fixture corpus under tests/fixtures/golden_regression/, indexed by a single repo-relative manifest, that freezes THREE seams along the trajectory-hybrid pipeline (the only video-free seams that exist): (1) the post-detector trajectory CSV `Frame,Visibility,X,Y` + the (fps, frame_width) it needs; (2) the per-signal feature streams (shuttle 5-col + Gate-A net_crossings, plus the default-OFF person 5-col) as a `\*\_golden\_features.json`; (3) an expected/snapshot final-segments file (the merged play-segment `(start,end)` time_pairs at a pinned windowing+operating point). An extraction CLI (`backend/eval/freeze\_fixture.py`) turns one labeled-and-detected video run into exactly these committed artifacts plus a per-video `provenance.json` (licensing/consent/minors-free assertion, detector + windowing operating point, fps/frame_width, schema_version, source hashes). Two CI-runnable gates re-use the SHIPPED scoring stack with ZERO new scoring math: (a) a CHARACTERIZATION gate that replays trajectory?windows?features?merge through `windowing.build\_windows` + `experiment.predict`/`merge\_overlaps` and asserts byte/float-tolerant exact match against the frozen `expected\_segments.json` + `expected\_features.json` (catches ANY refactor side-effect, no labels needed); (b) a QUALITY-FLOOR gate that scores the frozen substrate vs the committed golden labels via `metrics.score`/`experiment.compare` and asserts F1 stays at-or-above a committed floor tied to `eval\_baselines/regression\_baseline.json` and `QUALITY\_ITERATIONS`, with the honest ?F1 CI + sign-test wired in. Both run as a single new `golden-regression` CI job in the no-GPU lane. The corpus stays git-sized (numeric-only, low single-digit MB), so no git-lfs. CRITICAL PRECONDITION: committing proprietary-derived per-video artifacts reverses the GOLDEN_DATA_SHARING.md private-OneDrive posture ? this is owner-gated; the design ships SYNTHETIC + (owner-cleared, minors-free, de-identified numeric) fixtures only.",

"buildsOnExisting": [
 "backend/eval/metrics.py ? interval_iou/match_intervals/score (greedy, deterministic, 0.0-not-NaN on empty); the ONLY scoring math, reused verbatim by both gates",
 "backend/eval/gt_loader.py::load_gt_intervals ? the one canonical golden-CSV reader (header start,end | start_time,end_time | positional; MM:SS or decimal; strict raises file:line); committed label CSVs must conform",
 "backend/eval/regression.py ?
 regress()/RegressionResult/gt_hash/save_baseline/load_baselines + the
 DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json convention + exit 0/1/3 contract +
 incommensurability guards (IoU/detector/gt_hash mismatch -> flagged not silently compared)",
 "backend/eval/experiment.py ?
 VideoCandidates/Variant/predict/compare/evaluate_variant/CONTROL + record_experiment +
 DEFAULT_LEADERBOARD_PATH + spec_hash/label_set_hash; the variant predict() is the deterministic
 gate->merge re-scoring used by the characterization gate",
 "backend/eval/windowing.py ? WindowingPreset/SERVED(from IndexingConfig
 defaults)/PROPOSAL/build_windows/windows_to_intervals/served_windowing_from_config; the single-
 sourced operating point the fixture pins (closes the eval!=serve drift)",
 "backend/eval/fusion_compare.py::load_videos ? the canonical glob+json.load of
 *_golden_features.json into VideoCandidates; the loader the characterization+quality gates
 mirror but pointed at COMMITTED files",
 "backend/eval/distill_local.py::build_candidates + baseline_windows ? trajectory CSV ->
 (intervals, 5 shuttle feature rows) under a named preset; the exact transform the extraction CLI
 calls to produce the frozen feature stream",
 "backend/pipeline/detectors/tracknet_runner.py ? parse_trajectory_csv +
 trajectory_to_action_windows (the pure 'windowing brain' under characterization) +
 TrajectoryPoint",
 "backend/pipeline/detectors/rally_gate.py + fusion_features.py ? net_crossings (Gate A,
 always) + PERSON_FEATURE_NAMES (5, default-OFF); the per-signal feature producers, all pure
 given boxes",
 "backend/eval/eval_stats.py (paired_delta_ci, mean_with_ci, sign test) +
 segmentation_metrics.py (f1_at_overlaps, segment_count_ratio) ? the 'NUMBERS INFORM' honest-
 stats layer the quality gate emits",
 "backend/eval/served_gate_a.py + tests/test_served_gate_a_regression.py ? the manifest-
 driven, skip-if-absent per-video regression template to mirror (env-overridable MANIFEST,
 _golden_specs() presence filter, _skip decorator), upgraded to run-in-CI because fixtures are
 committed",
 "eval_baselines/ ? the proven TRACKED home for committed eval score artifacts
 (heuristic_lovo_n6.json, experiment_leaderboard.json); regression_baseline.json default path
 reserved but not yet present",
 "backend/infrastructure/database.py PRAGMA user_version ladder + the detector-agnostic
 'common columns + stats JSON keyed by detector string' schema pattern ? mirrored by the fixture
 schema_version + provenance design",

"run_all_tests.py + pytest.ini (testpaths=tests, no conftest, inline-tmp_path fixture idiom) + .github/workflows/ci.yml (lint/mypy/import-smoke/full-suite, 70% cov floor) ? the suite the new parametrized test and CI job slot into"

```
],
"seams": [
  "SEAM 1 (post-detector trajectory): the `Frame,Visibility,X,Y` CSV emitted by
  WASB/TrackNet run_predict ? frozen as `
```

backend.eval.freeze_fixture --manifest output/human_lovo_manifest.json --slug <slug> --out tests/fixtures/golden_regression/<slug> [--preset served] [--license owned --minors-free --consent-ref <doc>]`. It runs on the owner's box (or fully on CPU for synthetic) and, for one manifest entry: (1) copies/normalizes the trajectory CSV -> trajectory.csv and the golden labels -> labels.csv (projecting the rich Roora <video_id>.csv to the minimal gt_loader start,end shape, recording the projection in provenance); (2) calls distill_local.build_candidates(traj, fps, fw, preset) for intervals+shuttle features, rally_gate for net_crossings, and (only if --with-person and a video is present) fusion_golden.extract_video for the person 5-col, writing expected_features.json; (3) computes the snapshot by experiment.predict(CONTROL, VideoCandidates)->merge_overlaps at the pinned preset and writes expected_segments.json with the variant spec_hash + preset stamp; (4) emits provenance.json with gt_hash(labels) + label_set_hash + the exact freeze_command + the licensing/minors assertion (REFUSES to write unless --minors-free is passed and a --license is given, so an unvetted fixture cannot be frozen). It is a thin composition over SHIPPED functions (no new transforms), and is itself unit-tested (synthetic in -> deterministic artifacts out). A companion `--refresh-snapshot` mode re-derives ONLY expected_segments.json/expected_features.json from the committed trajectory+labels after an INTENTIONAL behaviour change, so re-baselining is one reviewed command (the characterization analogue of run_regression --update-baseline).",

"regressionGates": "DUAL gates, one new test module each, both no-GPU/CI-runnable, both parametrized over manifest.json via Path(__file__).parent.glob (mirroring fusion_compare.load_videos + test_served_gate_a_regression's per-video loop, but reading COMMITTED files so they actually run in CI). GATE (a) CHARACTERIZATION / snapshot (tests/test_golden_characterization.py): for each fixture, parse_trajectory_csv(trajectory.csv) -> windowing.build_windows(preset) -> build the VideoCandidates -> recompute features (shuttle via distill_local.build_candidates, net_crossings via rally_gate) and recompute segments via experiment.predict(CONTROL)->merge_overlaps; assert the recomputed features == expected_features.json and recomputed segments == expected_segments.json. NEEDS NO LABELS ? it catches ANY side-effect of a refactor to the windowing brain / feature math / merge, even one that doesn't change F1. Floats compared with pytest.approx(abs=1e-6) tolerance (the determinism contract: float ops are platform-stable but assert with tolerance, never exact ==); never assert reel bytes / DB ids / timestamps (frozen seam stops before those). A mismatch is a HARD FAIL with a diff, forcing either a bug-fix or a reviewed --refresh-snapshot. GATE (b) QUALITY-FLOOR (tests/test_golden_quality_floor.py): load the committed substrate via the fusion_compare.load_videos shape, score CONTROL vs labels with metrics.score per video and experiment.compare for the honest aggregate; assert per-video AND mean F1 >= the committed floor in eval_baselines/regression_baseline.json (created via freeze + regression.save_baseline, keyed <slug>::final with iou/detector/gt_hash stamps so a label or operating-point change is flagged incommensurable, exit-3 semantics surfaced as a test failure with a clear message). Emits eval_stats.paired_delta_ci + sign test in the assertion message (NUMBERS INFORM) and ties the floor doc-side to QUALITY_ITERATIONS. Tolerance = regression.DEFAULT_TOLERANCE (0.05) below floor trips it; structural guards (Gate A) are never auto-demoted on a small-n null.",

"repoSizeStrategy": "Numeric-only, git-sized, NO git-lfs. Trajectory CSV is the only large artifact: ~20-25 bytes/row x one row/frame; a deliberately SHORT fixture clip (synthetic ~30-90s, or owner-cleared real clips trimmed to a few rallies) keeps each CSV well under ~200KB; expected_features.json/expected_segments.json/provenance.json are KB-scale. A 2-3 video synthetic tier-0 corpus is < ~500KB total; even a full owner-cleared n=6 numeric set is < ~2MB ? comfortably under git's comfort zone, so the repo's established 'keep big data out of git, derived numbers are fine' line holds and .gitattributes/lfs is NOT introduced (git-lfs is installed v3.5.1 but intentionally unused; this design keeps it that way). Guard against creep: a unit test asserts each committed trajectory.csv row-count <= a cap (e.g. 12000 rows) so nobody accidentally commits a full 4K-match trajectory; the freeze CLI warns past the cap. Fixtures are tests/ only (not packaged into the wheel ? pyproject packages=[backend,training] ? which is correct, tests aren't shipped). output/ remains gitignored and the OneDrive sync remains the home for the FULL proprietary corpus; the repo holds only the small committed regression subset.",

"licensingHandling": "This is the load-bearing constraint and an OWNER-GATED reversal of GOLDEN_DATA_SHARING.md (which says proprietary-derived per-video artifacts live in private OneDrive, never committed). Reconciliation: (1) tier-0 SYNTHETIC fixtures (hand-authored trajectories, no real footage, no players) are licensing/privacy-clean by construction and ship WITHOUT an owner gate ? they make the gates CI-real immediately. (2) tier-1 real fixtures are added ONLY on explicit owner sign-off and ONLY as DERIVED de-identified NUMERIC streams (frame index, pixel coords, scale/translation-invariant scalar features, interval labels) ? never frames/imagery, so no biometric/identifiable content; sourced only from owned/licensed, MINORS-FREE footage (founder collection / ShuttleSet), with each fixture carrying a provenance.json {license, owned_or_synthetic, minors_free:true, consent_ref} assertion. The freeze CLI REFUSES

to write without `--license + --minors-free`, giving a code-enforced gate where the consent ledger is still only a convention/BACKLOG. Detectors producing the streams (WASB/TrackNet) are MIT, and NO paid-LLM (Gemini) output is committed as a training/data source ? labels are human/owned (Roora work-for-hire) or synthetic. The PR that introduces tier-1 must cite the owner approval; the design ships tier-0 first so the framework lands without waiting on that decision.",

"determinismHandling": "Freeze ONLY at the pure-Python seams BEFORE every non-deterministic step. Controlled: (1) FLOATS (velocity = dist/frame_width * fps/dframes; rally_gate median/spread; normalized person motion) ? platform-stable but assert with `pytest.approx(abs=1e-6)`, never exact ==. (2) fps/frame_width are NOT in the CSV (probed from video via cv2) ? they are PINNED in provenance.json + each feature/segment file's windowing block, so a replay never hits the silent 30.0/1280.0 fallback. (3) windowing operating point is pinned to a named WindowingPreset (default SERVED, single-sourced from IndexingConfig) and stamped into expected_segments.json ? the eval=serve cautionary tale (Gate-A +0.074 proposal vs -0.008 served) is neutralized by recording WHICH windowing the snapshot was taken at; a deliberate retune trips the incommensurability guard, not a silent pass. EXCLUDED entirely (never frozen/asserted): provider/Gemini output and ThreadPool ordering (seam stops before the AI handoff ? CONTROL emits windows verbatim, skip_ai path); DB autoincrement ids + created_at timestamps (compared on (start,end,stats) not id); ffmpeg/codecs reel bytes (assert time_pairs + merge output, never bytes). The motion + gemini segmenters (random metadata / network) are explicitly out of scope ? only the trajectory-hybrid family is freezable. Person per-frame detections (cv2 frame-seek ±frames by codec) are frozen as the captured dict from the SAME decode, default-OFF, and the synthetic tier hand-authors them so CI is fully deterministic.",

"versioningStrategy": "schema_version int on every committed JSON (starts at 1), mirroring the DB PRAGMA user_version migration-ladder discipline. The loader (a small ``backend/eval/fixtures.py`` module: `load_manifest + load_fixture`, the shared plumbing both gate tests import) dispatches on schema_version with ordered ``if version < N`` upgrade shims that normalize an old fixture into the current in-memory shape (additive fields default safely ? same as `experiment._feature_column` defaulting an absent column to 0.0), so an old committed fixture keeps loading after the schema grows. A unit test asserts every committed fixture's `schema_version` is `<=` the loader's `CURRENT_SCHEMA` (drift fails CI, exactly as the DB version assertion does). Schema changes are additive-only (new feature columns, new provenance fields) ? never a breaking rename ? so the characterization gate stays valid across versions; a genuinely breaking change requires re-freezing all fixtures in the same PR (the freeze CLI's `--refresh-snapshot` batch mode) and bumping `CURRENT_SCHEMA`. The feature/preset/variant identity travels via the already-shipped fingerprints (`gt_hash`, `label_set_hash`, `Variant.spec_hash`, `WindowingPreset name+tuple`) so any baseline/snapshot comparison against changed labels/recipe/windowing is flagged incommensurable rather than silently trusted.",

"ciIntegration": "Add one new job ``golden-regression`` to `.github/workflows/ci.yml` in the NO-GPU lane (the gates are pure-CPU; cv2 is only needed transitively by the served engine import, so the tests `pytest.importorskip('cv2')` OR the gate logic is refactored to import only the pure windowing/metrics path ? preferred, so it runs even without cv2). Job: `checkout -> setup-python 3.13 -> pip install pytest + minimal pure deps (pydantic for IndexingConfig defaults) -> `python -m pytest tests/test_golden_characterization.py tests/test_golden_quality_floor.py -v``. It is ALSO collected by the existing full-suite job (`testpaths=tests`) and contributes to the 70% coverage floor, but a dedicated job gives a clear red/green signal and lets it run without the torch/cv2 install the full-suite needs. The synthetic tier guarantees the job has fixtures to run in CI (the gap today: `test_served_gate_a_regression` skips because the corpus is uncommitted). Optionally, a SEPARATE scheduled/owner-box workflow runs `run_regression.py` against the real DB for the full proprietary corpus (the existing exit 0/1/3 path) ? that stays out of PR CI. No new marker is registered (rely on `parametrize + path-glob`, matching the zero-conf test, zero-custom-marker convention) to avoid `PytestUnknownMarkWarning`.",

"contributorWorkflow": "Adding/refreshing a fixture is a reviewed, scripted path. To ADD a video: `run `python -m backend.eval.freeze_fixture --manifest <local> --slug <slug> --out tests/fixtures/golden_regression/<slug> --preset served --license owned --minors-free --consent-ref <doc>`` (owner box for real, any box for synthetic) -> it writes the 5 artifacts + appends to `manifest.json` -> contributor commits the explicit paths (NEVER `git add -A` ? shared-checkout safety) on a branch cut from origin/master, re-verifying HEAD before commit. To accept an INTENTIONAL behaviour change (the windowing brain or feature math legitimately changed): `run `freeze_fixture --refresh-snapshot`` (regenerates `expected_features.json + expected_segments.json` from the committed trajectory+labels, NO re-detection needed since the trajectory is frozen) and, if quality moved, `run `run_regression.py --update-baseline`` to re-snapshot the floor in `eval_baselines/regression_baseline.json` ? both land in the SAME PR as the code change, with the diff reviewed. The PR description must state which gate moved and why (a characterization diff is mechanical; a floor change cites the `QUALITY_ITERATIONS` ablation). Rollback of a bad change = revert the code; the fixtures/baseline are the source of truth, never hand-edited. README.md in

the fixtures dir documents this loop + the minors/licensing guardrail so a future contributor cannot freeze an unvetted video.",

"pros": [

- "Closes the real gap: for the FIRST time a fresh git clone / CI can run an end-to-end per-video rally regression with no GPU/WSL/video/DB/network ? today every such test skips because the corpus is uncommitted (test_served_gate_a_regression) or lives in gitignored output/",
- "Reuses the ENTIRE shipped scoring/loader/stats/gate stack verbatim (metrics, gt_loader, regression, experiment, windowing, eval_stats, fusion_compare.load_videos) ? adds committed inputs + a thin loader + two test modules + one freeze CLI, no new scoring math to maintain or get wrong",
- "Three independent frozen seams catch DIFFERENT regressions: characterization catches any refactor side-effect even at constant F1 (windowing brain, feature math, merge); quality-floor catches a true quality drop; per-signal feature freezing catches a Gate-A/person-feature drift that segment-level snapshots would miss",
- "Determinism handled honestly: pins fps/frame_width/windowing-preset (kills the silent 30/1280 fallback and the eval!=serve drift that burned Gate-A), freezes before all non-deterministic steps (provider/ThreadPool/DB ids/ffmpeg bytes), float-tolerant asserts ? so the gates are stable, not flaky",
- "Stays git-sized and lfs-free with a row-count cap test guarding creep; honors the 'derived numbers OK, frames/weights never' line the repo already draws",
- "Versioned + fingerprinted for longevity: schema_version ladder + gt_hash/label_set_hash/spec_hash/preset stamps mean an old fixture keeps loading and any incommensurable comparison is flagged, not silently trusted",
- "Synthetic tier-0 lands the whole framework WITHOUT waiting on the owner privacy decision, so the gates are CI-real on day one; real tier-1 is purely additive on sign-off",
- "Extensible: a new cue (audio, future shuttle-drop) is just a new named feature block in expected_features.json + a Variant feature_name ? the same gates score it default-OFF with zero regression, matching the weak-signals-retained directive"

],

"cons": [

- "Higher up-front implementation cost than a single-gate snapshot: a freeze CLI + a shared fixtures loader + two parametrized test modules + a CI job + schema-version plumbing + the synthetic corpus ? deliberately chosen for longevity over a quick win",
- "Tier-1 (real-footage-derived) fixtures REQUIRE an owner decision that reverses GOLDEN_DATA_SHARING.md's 'never commit proprietary-derived artifacts' posture; until then the gates run only on synthetic data, which exercises the PLUMBING faithfully but not real-footage edge cases",
- "Synthetic trajectories may not reproduce real-footage failure modes (occlusion breaks, codec frame-seek jitter), so the quality-floor gate's numbers on tier-0 are about pipeline correctness, not field quality ? the real quality signal still needs the owner-box run_regression against the proprietary DB",
- "Characterization snapshots must be re-frozen on any INTENTIONAL behaviour change ? a small recurring maintenance tax (mitigated by --refresh-snapshot, but a sloppy contributor could rubber-stamp a real regression by refreshing without review)",
- "Freezing the person 5-col seam requires a captured per-frame detection dict (cv2-decode dependent) for real fixtures ? only fully deterministic for hand-authored synthetic; real person-feature fixtures carry codec-seek caveats noted in provenance",
- "Two sources of truth still coexist (DB-backed run_regression vs file-backed gates); this design picks the trajectory/feature substrate and records its provenance, but a contributor must understand the DB path is a separate, owner-box-only gate"

],

"implementationSteps": [

- "1. Land the framework PR-1 (no owner gate needed): create tests/fixtures/golden_regression/ + README.md (guardrails + freeze loop) + backend/eval/fixtures.py (load_manifest/load_fixture, schema_version dispatch, Path(__file__)-relative resolution, CURRENT_SCHEMA=1).",
- "2. Author 2-3 SYNTHETIC tier-0 fixtures by hand (tiny trajectory.csv + labels.csv covering: a clean rally, a held-shuttle/service window Gate-A should cut, and an occlusion break) and run the freeze CLI to emit their expected_features.json/expected_segments.json/provenance.json so the committed artifacts are tool-generated, not hand-typed.",
- "3. Write backend/eval/freeze_fixture.py composing distill_local.build_candidates + rally_gate + experiment.predict(CONTROL)+merge_overlaps + regression.gt_hash; enforce --license/--minors-free refusal; add --refresh-snapshot; unit-test it (synthetic in -> deterministic artifacts out).",

"4. Write tests/test_golden_characterization.py ? parametrize over manifest.json, recompute features+segments from the frozen trajectory under the pinned preset, assert `pytest.approx(abs=1e-6)` match vs `expected_features.json/expected_segments.json`; assert no DB-id/timestamp/byte fields are touched.",

"5. Write tests/test_golden_quality_floor.py ? load the substrate via the `fusion_compare.load_videos` shape, score CONTROL vs labels with `metrics.score + experiment.compare`, assert `per-video+mean F1 >=` the committed floor; emit `eval_stats` CI + sign test in the failure message; create `eval_baselines/regression_baseline.json` keyed `<slug>::final` via `regression.save_baseline` with `iou/detector/gt_hash` stamps.",

"6. Add a schema-version drift test (every fixture `schema_version <= CURRENT_SCHEMA`) and a trajectory row-count cap test (repo-size guard).",

"7. Add the golden-regression CI job to `.github/workflows/ci.yml` (no-GPU lane, minimal deps, pytest the two new modules); confirm green locally via `python run_all_tests.py` and that coverage stays `>= 70%`.",

"8. Document the framework in `docs/` (a new `GOLDEN_REGRESSION_FIXTURES.md` or a section in `EVALUATION.md/EXPERIMENT_HARNESS.md`) and tie the quality floor to `QUALITY_ITERATIONS`; update `CODE_MAP §0` file-placement table with the new tests/fixtures/golden_regression/ home.",

"9. (Owner-gated, separate PR-2) On explicit sign-off, add tier-1 owner-cleared minors-free de-identified real numeric fixtures via the freeze CLI, citing the approval + reconciling `GOLDEN_DATA_SHARING.md` (note the committed subset vs the OneDrive full corpus).",

"10. Branch hygiene throughout: `git fetch origin`; `git checkout -b <name> origin/master`; stage explicit paths only (no `git add -A`, avoid the untracked artifacts/ dir + weights); re-verify HEAD before commit; one PR per work item, no stacking; do not open the PR until the owner asks."

```
]
},
{
  "approachName": "Characterization snapshot of the deterministic post-trajectory pipeline,
frozen at the committed trajectory-CSV seam",
  "summary": "Commit a tiny, video-free golden fixture set (per-video shuttle trajectory CSV +
fps/frame_width + GT rally CSV, plus optional frozen person-detection dicts for fusion) under
tests/fixtures/golden/, and a single parametrized characterization test that replays the
EXISTING pure stack ? TrackNetRunner.parse_trajectory_csv -> trajectory_to_action_windows ->
rally_gate -> metrics/segmentation_metrics ? and asserts the resulting candidate windows + gated
handoff windows + P/R/F1 against a committed `expected_snapshot.json`. The reviewer's guarantee:
any refactor that changes deterministic behavior between the trajectory CSV and the final time-
pairs makes the snapshot diff non-empty, so a side effect cannot land silently. A `--update-
snapshot` workflow regenerates the JSON so an INTENTIONAL behavior change shows up as a
reviewable snapshot diff in the same PR. The test runs in CI's existing no-GPU lane (no video,
no torch/cv2 needed for the trajectory-hybrid math path, no DB). This is characterization
(snapshot) testing, NOT correctness testing ? it locks current behavior, with explicit,
documented limits.",
  "buildsOnExisting": [
    "backend/eval/metrics.py ? score()/match_intervals() greedy deterministic matcher
(TP/FP/FN, empty->0.0 not NaN); the scoring math is reused verbatim, no new scoring code",
    "backend/eval/segmentation_metrics.py ? f1_at_overlaps/segment_count_ratio for over-
segmentation sensitivity in the snapshot",
    "backend/pipeline/detectors/tracknet_runner.py ? parse_trajectory_csv
(Frame,Visibility,X,Y) + trajectory_to_action_windows: the pure windowing 'brain' that is the
unit-under-characterization",
    "backend/pipeline/detectors/rally_gate.py ? Gate-A net_crossings (pure, from trajectory)
for the fusion handoff-selection snapshot",
    "backend/eval/calibrate_wasb.py::make_predict_fn ? the EXISTING proof that a cached
trajectory CSV replays windowing with no video; the new test reuses this exact pattern from
committed files",
    "backend/eval/gt_loader.py::load_gt_intervals ? the one canonical GT CSV reader; committed
GT fixtures conform to its start,end / start_time,end_time conventions",
    "backend/eval/regression.py ? gt_hash() for label-fingerprinting, the eval_baselines/
tracked-artifact convention, and the exit 0/1/3 gate contract (reused for an optional CLI), plus
its incommensurability-guard idea (record what the snapshot was frozen at)",
    "tests/test_served_gate_a_regression.py ? the per-video, manifest-driven, skip-if-absent
loop is the structural template; the new test differs only in that fixtures are COMMITTED so it
actually runs in CI",
    "tests/test_fusion_compare.py + backend/eval/fusion_compare.py::load_videos ? the
canonical glob+json.load 'load artifact -> transform -> assert' shape (today on tmp_path; the
```

```

new test reads committed files instead)",
    "eval_baselines/heuristic_lovo_n6.json ? the precedent for a committed JSON carrying
name/description/`computed` repro command + provenance; expected_snapshot.json mirrors this
provenance discipline",
    "run_all_tests.py + pytest.ini (testpaths=tests, auto-discovery, no conftest needed) ? the
test slots straight into the existing offline/mockd suite under the 70% coverage floor"
],
    "seams": [
        "PRIMARY freeze seam = the trajectory CSV (Frame,Visibility,X,Y) per video. This is the
exact handoff from the GPU/WSL world to pure Python; everything downstream (windowing, net-
crossing Gate A, candidate stats, IoU scoring, merge) is deterministic stdlib math. Freezing
here characterizes the largest deterministic surface ? the windowing 'brain' itself ? which a
downstream candidate-row snapshot could NOT (it is already one transform past windowing).",
        "REQUIRED side-inputs frozen alongside the CSV: (fps, frame_width).
trajectory_to_action_windows normalizes velocity by frame_width and converts frame->sec by fps;
_probe_fps_width reads these from the video via cv2 with SILENT fallbacks (fps<=0 -> 30.0,
width<=0 -> 1280.0). A video-free replay that forgets to pin them silently uses fallbacks and
masks regressions ? so each fixture's manifest records the exact (fps, frame_width) used.",
        "GT seam = a per-video golden rally CSV in gt_loader conventions (start,end or
start_time,end_time; decimal or MM:SS). Loaded via load_gt_intervals(strict=True) so a malformed
committed label fails loudly with file:line.",
        "DOWNSTREAM ASSERTION seams the snapshot pins: (a) candidate windows list =
trajectory_to_action_windows output at the pinned production operating point (vel 0.02, merge
2.0, min 2.0); (b) per-window net_crossings + the Gate-A-selected handoff index set (fusion
path, min_crossings configurable); (c) the metrics.score result (precision/recall/f1/mean_iou)
and segmentation_metrics (f1@k, segment_count_ratio) of candidate intervals vs the committed GT.
These three are the deterministic 'observable behavior' a refactor must preserve.",
        "STUBBED / EXCLUDED seams (where the snapshot deliberately stops): the AI/provider handoff
(provider.analyze_video is network/non-deterministic) and the ffmpeg stitch (compiler re-encode
bytes are not reproducible across ffmpeg/NVENC builds). The snapshot freezes the time-pairs that
ENTER the stitch and the merge output of _merge_overlapping (pure), never reel bytes. The
provider path is characterized only via the pre-handoff window list + a separately stubbed
deterministic provider if/when desired."
    ],
    "fixtureLayout": "\"NEW tracked dir tests/fixtures/golden/ (there is no tests/fixtures
today; adding it is consistent with CODE_MAP $0 'a test -> tests/'). Per fixture video (use
SHORT synthetic or owner-cleared real-derived clips), one subdir <case>/ containing:
trajectory.csv (Frame,Visibility,X,Y), labels.csv (gt_loader convention), and meta.json {case,
fps, frame_width, source, license, consent, minor_free:true, schema_version}. A top-level
tests/fixtures/golden/manifest.json lists the cases (repo-RELATIVE paths, resolved via
Path(__file__).parent ? NEVER cwd, since agent/test cwd is not stable) and the frozen operating
point. The committed expected output lives next to each case as expected_snapshot.json (or one
consolidated tests/fixtures/golden/expected_snapshot.json keyed by case). For the fusion/person
path (default-OFF), add an optional person_detections.json per case
({frames:{idx:[[track_id,x1,y1,x2,y2]], fps, frame_width, frame_height}) frozen from the SAME
decode as the trajectory so frame indices align 1:1. Keep each fixture tiny (a handful of
windows; trajectory a few hundred rows). The new test = tests/test_golden_characterization.py
globbing manifest.json. No conftest.py needed (module-local glob matches the zero-conftest
convention); add tests/conftest.py only if multiple test files later share the loader.\""),
    "artifactSchema": "\"Two committed JSON shapes, both carrying provenance like
eval_baselines/heuristic_lovo_n6.json. (1) Per-case INPUT meta.json: {schema_version:int,
case:str, fps:float, frame_width:float, operating_point:{velocity_thresh:0.02, merge_gap:2.0,
min_window_duration:2.0}, source:str, license:'owned|ShuttleSet|synthetic', consent:str,
minor_free:true, gt_hash:<gt_loader gt_hash of labels.csv>}. (2) Committed EXPECTED
expected_snapshot.json per case: {schema_version, case, generated:'<ISO date>',
generator_cmd:'python -m tests... --update-snapshot', operating_point, fps, frame_width,
gt_hash,
candidate_windows:[{start,end,active_frame_count,peak_velocity,mean_velocity,net_crossings}],
handoff_indices_gateA:{min_crossings:2 -> [int,...]},
score_vs_gt:{precision,recall,f1,mean_iou,num_pred,num_gt,tp,fp,fn},
segmentation:{f1_at_overlaps:{'0.1':...,'0.25':...,'0.5':...}, segment_count_ratio}}. Floats are
stored ROUNDED to a fixed precision (e.g. 6 dp) so the JSON is byte-stable; assertions compare
with pytest.approx tolerance, never exact float equality. The detector tag + operating_point +
gt_hash travel in the snapshot so a comparison against a changed operating point or changed
labels is flagged incommensurable (mirroring regression.py's guard) rather than silently

```


'passing'.\'",

"extractionMechanism": "\"A small extractor (tests/golden/extract.py or a backend/eval/freeze_golden.py CLI) that, GIVEN a committed case dir, runs the SAME pure functions the test runs and writes expected_snapshot.json: parse_trajectory_csv -> trajectory_to_action_windows(operating_point) -> rally_gate.estimate_play_axis + gate_windows for net_crossings/handoff indices -> load_gt_intervals(labels) -> metrics.score + segmentation_metrics. Crucially the EXTRACTOR and the TEST call ONE shared helper (e.g. tests/golden/_replay.py::replay_case(case_dir)->snapshot_dict) so the generated artifact and the asserted artifact are computed by identical code ? the test asserts replay_case(case)==committed_snapshot. Producing the trajectory.csv/labels.csv inputs themselves is a SEPARATE, owner-box, one-time step (the real GPU/WSL pass writes the trajectory; the vendor/owner writes labels), out-of-band from CI; CI only ever replays committed inputs. The update workflow re-invokes the same extractor: `python -m tests.golden.extract --update-snapshot [--case X]`.\""

"regressionGates": "\"In CI (full-suite lane), tests/test_golden_characterization.py is a normal pytest that, per case, computes replay_case() and asserts it equals the committed expected_snapshot.json (floats via approx, lists exact in length/order). A behavioral side effect anywhere in the frozen seam -> assertion fails -> the PR's CI is red. The contributor cannot merge without either (a) the change being behavior-preserving (snapshot still matches) or (b) deliberately running --update-snapshot and committing the new snapshot, which surfaces as a diff the reviewer reads. Distinct exit semantics for an optional standalone CLI mirror run_regression.py (0=match, 1=snapshot drift, 3=no-snapshot-to-compare so CI never silently green-lights a missing baseline). The gate is purely the snapshot-equality test ? no statistical promotion/CI logic is needed here (that lives in the experiment/ablation track and is orthogonal to this characterization gate).\""

"repoSizeStrategy": "\"Commit numeric-only artifacts, no frames/video, no *.db, no weights. Size budget is comfortably git-native: a trajectory CSV is ~20-25 bytes/row; a deliberately SHORT fixture clip (10-30s) is a few hundred to ~2k rows = single-digit to low-tens of KB; labels.csv + meta.json + expected_snapshot.json are sub-KB to a few KB each. Even a full n=6-style set of short clips is well under ~1-2 MB total. git-lfs is installed (3.5.1) but UNUSED (no .gitattributes) and is NOT needed at this scale ? keep the repo's 'small numeric artifacts in git, bulk data out' posture. Prefer 3-5 SHORT cases over long real matches: characterization wants behavioral coverage of code paths (a quiet trajectory, a high-velocity rally, an occlusion gap that resets the track, a held-shuttle stretch that exercises Gate A), not long footage. Keep fixtures minimal and add cases only to cover a new code path.\""

"licensingHandling": "\"Commit ONLY derived numeric streams (shuttle X/Y trajectory rows, interval labels, scale/translation-invariant feature/feature-count JSON) ? never frames, never imagery, so no biometric/identifiable content and no paid-LLM output (the streams come from MIT-licensed WASB/TrackNet detectors, never from Gemini). EACH fixture's meta.json carries an explicit license + consent + minor_free:true assertion. PREFER SYNTHETIC trajectories (hand-authored Frame,Visibility,X,Y that exercise each windowing/Gate-A branch) as the default ? synthetic data is unambiguously licensing-clean, privacy-clean, and minors-free, and is sufficient for CHARACTERIZATION (we are locking code behavior, not validating real-world accuracy). Real-derived trajectory fixtures require explicit owner sign-off because GOLDEN_DATA_SHARING.md currently keeps proprietary-derived artifacts OUT of git (private OneDrive). This design REVERSES that convention for a tiny vetted subset, so it is an owner-gated product/privacy decision ? flag it, do not land real-derived fixtures unilaterally. Minors are excluded at source; never commit any artifact derived from footage with minors.\", \"Honest caveat: even derived trajectories encode something about proprietary footage. The synthetic-first default sidesteps this entirely and is the recommended path; real-derived fixtures are an owner call.\""]"

"determinismHandling": "\"The frozen seam is pure stdlib float math (no numpy/torch/cv2 in the trajectory-hybrid windowing path; no DB; no network), which is the precondition for cross-machine stability. Controls: (1) FLOATS ? velocity=(dist/frame_width)*(fps/dframes), rally_gate medians, F1 are IEEE-754 double ops; store snapshot floats rounded to fixed precision and assert with pytest.approx(abs=1e-6), never exact equality, so last-ULP cross-OS/CPython-build differences don't flap. (2) ORDERING ? match_intervals sorts candidates by (-iou, pi, gi) (deterministic tie-break) and windowing emits groups in frame order; both are stable, so list order in the snapshot is reproducible. (3) LINE ENDINGS ? commit a .gitattributes entry so tests/fixtures/golden/* CSV/JSON are checked out with LF on every OS (Windows-vs-WSL/Linux), and open CSVs with newline='' (parse_trajectory_csv already does) so CRLF/LF never changes parsed rows; this is the one new .gitattributes use and it is the cross-OS linchpin. (4) ENCODING ? gt_loader reads utf-8-sig (BOM-tolerant); keep fixtures UTF-8. (5) EXCLUDE non-deterministic columns entirely from the snapshot ? no DB autoincrement id, no created_at timestamps, no reel bytes, no provider output. (6) PIN the operating point and (fps,frame_width) in the snapshot so

a silent fallback (30.0/1280.0) or a retune is caught, not absorbed. Net: identical bytes in (committed CSV+meta) -> identical snapshot on Windows, WSL, and Linux CI within tolerance.\",
\"Residual risk to state plainly: pytest.approx tolerance is a small blind spot ? a refactor that perturbs a float below tolerance won't trip; choose abs=1e-6 (far below any behaviorally meaningful change in seconds/F1) so this is negligible, and keep integer/structural fields (window counts, handoff index sets, tp/fp/fn) as EXACT asserts since those carry the behavioral signal.\",

\"versioningStrategy\": \"Each artifact carries schema_version:int (mirroring the DB PRAGMA user_version ladder discipline). When the snapshot schema changes (a new asserted field), bump schema_version and migrate all committed snapshots in the same PR; the test refuses to compare across schema_version mismatch (treats it as drift requiring --update-snapshot, like regression.py's incommensurability guard). The operating_point + gt_hash + detector tag embedded in every snapshot bind it to the exact recipe and labels it was frozen at, so changing the production operating point or relabeling forces a deliberate, reviewable re-freeze rather than a silent mismatch. Inputs (trajectory.csv/labels.csv) are immutable per case once committed; a genuinely new behavioral scenario is a NEW case dir, not an edit, so history stays clean and a diff to an existing case is always either a fixture-input change (rare, reviewed) or a snapshot re-freeze (the intentional-change signal).\",

\"ciIntegration\": \"Lands in the EXISTING full-suite lane (.github/workflows/ci.yml) ? no new job needed; it is just another offline pytest under run_all_tests.py with the 70% coverage floor, and it ADDS coverage to the windowing/gate/metrics core (which today is only exercised by tmp_path synthetic tests or skipped golden tests). The trajectory-hybrid windowing math path does not require torch/cv2; if a chosen assertion transitively imports cv2 (as test_served_gate_a does via trajectory_hybrid), gate that import with pytest.importorskip('cv2') OR ? preferred ? call the pure staticmethods (parse_trajectory_csv, trajectory_to_action_windows, rally_gate, metrics) directly so the test runs even in the import-smoke/no-GPU lanes. Optionally also expose a standalone `python -m tests.golden.check` (exit 0/1/3) for a pre-PR local gate, but the pytest assertion IS the CI gate. No CI secret, no GPU, no network.\",

\"contributorWorkflow\": \"CONTRIBUTOR (other machine, no video): clones, runs `python run\_all\_tests.py`. If their refactor preserved behavior, the golden characterization test passes ? they push with confidence. If it fails, the diff in the assertion shows exactly which case/field changed (e.g. handoff_indices_gateA changed for case 'held_shuttle'). They then decide: (a) it's an unintended side effect -> fix the code until the snapshot matches again; or (b) it's an intended behavior change -> run `python -m tests.golden.extract --update-snapshot`, which rewrites the expected_snapshot.json files, and they COMMIT those snapshot changes in the same PR with a message explaining why. The contributor never needs video, GPU, WSL, or the proprietary corpus. REVIEWER: reads the PR diff. If expected_snapshot.json is UNCHANGED, the change is provably behavior-preserving across the frozen seam (within the documented limits) and the reviewer trusts the suite caught side effects. If expected_snapshot.json IS changed, the reviewer sees the precise before/after numbers (windows, indices, P/R/F1) in the diff and can judge whether the behavior change is intended and acceptable ? the change is explicit and reviewable, not buried. This is exactly the reviewer-trust UX the goal asks for.\",

\"pros\": [

\"Directly serves the stated goal: a reviewer of a no-video refactor PR gets a red/green signal that deterministic pipeline behavior was preserved, and any intentional change is a visible snapshot diff ? trust without re-running the GPU pipeline.\",

\"Reuses the entire existing pure stack (metrics, windowing, gt_loader, rally_gate, segmentation_metrics) verbatim ? no new scoring math, low surface area, low risk of the harness itself being wrong.\",

\"Runs in CI today (offline, no GPU/torch/cv2/DB/network), ADDS coverage to the windowing/gate core, and finally lets a golden-litmus actually execute in CI (today test_served_gate_a skips there).\",

\"Freezes at the trajectory-CSV seam ? the widest deterministic surface ? so it characterizes the windowing 'brain', not just a downstream slice.\",

\"Tiny, git-native, licensing/privacy-clean if synthetic-first (no frames, no minors, no paid-LLM output); no git-lfs needed.\",

\"Cross-OS stable by construction: pure float math + LF-pinned fixtures + approx tolerance on floats, exact on structural fields.\",

\"The --update-snapshot workflow makes intentional behavior changes explicit and self-documenting in the PR diff ? the core review affordance.\"

],

\"cons\": [

\"FALSE SAFETY (state loudly): a green snapshot proves only that DETERMINISTIC POST-TRAJECTORY behavior is unchanged for the FROZEN cases. It does NOT cover: the detector itself

(WASB/TrackNet trajectory production ? that needs video/GPU), video-I/O/decode (_probe_fps_width, frame seeking, ffmpeg), the AI/provider handoff (stubbed/excluded), reel-byte output, or any path the fixtures don't exercise. A refactor that breaks decoding or the detector passes this gate green.",

"Characterization, not correctness: the snapshot encodes CURRENT behavior including any current bugs; a 'passing' test means 'behavior unchanged', never 'behavior correct'. Easy to misread as a quality gate.",

"Coverage is only as good as the fixture cases ? an untested branch (a windowing edge case no fixture hits) has zero protection; needs discipline to add a case when a new code path appears.",

"--update-snapshot can be abused: a contributor can rubber-stamp a real regression by regenerating the snapshot. Mitigation is social/review (the diff is visible and must be justified), not technical ? be honest that the gate's strength depends on the reviewer actually reading snapshot diffs.",

"Real-derived fixtures conflict with the current GOLDEN_DATA_SHARING.md 'keep proprietary-derived data out of git' convention ? needs explicit owner sign-off; synthetic-first avoids this but synthetic trajectories may not exercise real-footage quirks.",

"approx tolerance is a (small, bounded) blind spot for sub-tolerance float drift."

```
],
"implementationSteps": [
  "1. Get OWNER sign-off on the fixture-source question: synthetic-first (recommended, no policy conflict) vs a tiny owner-cleared real-derived subset (reverses GOLDEN_DATA_SHARING.md ? owner-gated). Decide minor_free/license/consent assertions to embed.",
  "2. Add tests/fixtures/golden/ with 3-5 SHORT cases, each a subdir of trajectory.csv (Frame,Visibility,X,Y) + labels.csv (gt_loader convention) + meta.json; author synthetic trajectories that deliberately hit distinct branches: a quiet/no-active-frame trajectory, a clean high-velocity rally, an occlusion gap (visible=0 mid-rally resets prev), a held-shuttle stretch (low velocity, exercises Gate-A net_crossings), and a multi-window video. Add tests/fixtures/golden/manifest.json (repo-relative paths + frozen operating_point).",
  "3. Add a .gitattributes entry pinning tests/fixtures/golden/** to LF (text eol=lf) so CSV/JSON are byte-identical on Windows, WSL, and Linux CI.",
  "4. Write the shared replay helper
tests/golden/_replay.py::replay_case(case_dir)->snapshot_dict that calls parse_trajectory_csv -> trajectory_to_action_windows(operating_point) -> rally_gate (net_crossings + handoff indices) -> load_gt_intervals(strict=True) -> metrics.score + segmentation_metrics, rounding floats to fixed precision. This is the single source of truth for both generation and assertion.",
  "5. Write the extractor CLI tests/golden/extract.py (`--update-snapshot [--case X]`) that writes expected_snapshot.json via replay_case, carrying schema_version, generated date, generator_cmd, operating_point, fps, frame_width, gt_hash.",
  "6. Run the extractor once to produce and commit the initial expected_snapshot.json per case.",
  "7. Write tests/test_golden_characterization.py: glob manifest.json, parametrize over cases (ids=case name), assert replay_case(case)==committed snapshot ? exact on structural fields (window counts, handoff index sets, tp/fp/fn), pytest.approx(abs=1e-6) on floats; refuse cross-schema_version comparison. Call pure staticmethods directly (or importorskip('cv2') if a chosen assertion path needs it) so it runs in the no-GPU lane.",
  "8. Verify locally: `python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70` stays green and coverage does not regress.",
  "9. Write tests/fixtures/golden/README.md (or a docs/ section) documenting: what the snapshot DOES and does NOT guarantee (the false-safety limits), the exact --update-snapshot workflow, and the reviewer's read-the-diff responsibility.",
  "10. (Optional) add a standalone `python -m tests.golden.check` with exit 0/1/3 (match/drift/no-snapshot) mirroring run_regression.py for a pre-PR local gate; the pytest test remains the CI gate. Land as ONE PR branched from origin/master, staging explicit paths only."
]
}
```

2. assistant (2026-06-16T02:32:01.243Z)

I'll start by pulling the latest and grounding myself in the key existing modules and docs before synthesizing the unified design.

3. user (2026-06-16T02:32:04.170Z)

b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
master

4. assistant (2026-06-16T02:32:08.014Z)

Let me read the key modules and docs that the design hinges on, to ground the unified plan in real code.

5. user (2026-06-16T02:32:09.268Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here*; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.*
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
```

```

ratio
56     from backend.eval.score_calibration import fit_isotonic, fit_platt    # C2: calibrate cue
scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the control behaviour (no gate, no learned
82         filter), so a variant that merely carries a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105        """
106
107        name: str
108        segmenter: str = "wasb_hybrid"
109        config_overrides: Dict[str, Any] = field(default_factory=dict)
110        cue_flags: Dict[str, bool] = field(default_factory=dict)
111        min_crossings: int = 0
112        min_players: int = 0
113        classifier_model: Optional[str] = None
114        feature_names: Optional[List[str]] = None
115        threshold: float = 0.5
116        merge_gap: float = 1.0
117        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the

```

```

operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False           # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.
172         """
173
174         name: str
175         intervals: List[Interval]
176         features: Dict[str, List[float]] = field(default_factory=dict)
177         gts: Optional[List[Interval]] = None

```

```

178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215             model: Optional[LogisticRegression] = None,
216             threshold: Optional[float] = None,
217             calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
218     """Variant prediction: gate by persisted features, optionally filter by a learned
219     model, then merge. Pure and deterministic.
220
221     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
222     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
223     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
224     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
225     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
226     them.
227
228     """
229     mask = _gate_mask(variant, vc)
230     if variant.feature_names:
231         if model is None:
232             if variant.classifier_model is None:
233                 raise ExperimentError(
234                     f"variant {variant.name!r} is learned but has no model to predict "
235                     f"with (pass a fitted model or set classifier_model)")
236             model = LogisticRegression.load(variant.classifier_model)
237             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
238 variant.feature_names))]
239         if calibrator is not None:
240             proba = [calibrator(p) for p in proba]
241         thr = variant.threshold if threshold is None else threshold
242         mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]

```

```

239     kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240     return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                             train_videos: Sequence[VideoCandidates], iou: float
245                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                                                                 [])))
258                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259             if variant.calibrate == "platt":
260                 calibrator = fit_platt(raw, y)
261             elif variant.calibrate == "isotonic":
262                 calibrator = fit_isotonic(raw, y)
263             else:
264                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265                                     'platt'/'isotonic')")
266             threshold: Optional[float] = None
267             if variant.tune_threshold:
268                 best_t, best_f1 = variant.threshold, -1.0
269                 for k in range(1, 20):
270                     # grid 0.05..0.95 on the train
271                     t = round(0.05 * k, 2)
272                     f1s = [score(predict(variant, vc, model=model, threshold=t,
273                                         calibrator=calibrator),
274                               vc.gts or [], iou).f1 for vc in train_videos]
275                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276                     if mean_f1 > best_f1:
277                         best_f1, best_t = mean_f1, t
278                 threshold = best_t
279             return calibrator, threshold
280
281         # ----- variant scoring
282
283     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
284         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
285         the
286         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
287         X: List[List[float]] = []
288         y: List[int] = []
289         for vc in videos:
290             if vc.gts is None:
291                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
292             X.extend(_feature_matrix(vc, variant.feature_names or []))
293             y.extend(label_candidates(vc.intervals, vc.gts, iou))
294         if not X:
295             raise ExperimentError("cannot train a learned variant on zero candidates")
296         return LogisticRegression().fit(X, y, variant.feature_names)

```



```

297         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299     mean aggregate.
300
301     Prediction policy:
302     - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304     - **learned, pinned model** (`classifier_model` set): score every video with the
305     SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307     - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309     - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310     """
311     for vc in videos:
312         if vc.gts is None:
313             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315     per_video: List[Dict[str, Any]] = []
316     predictions: Dict[str, List[Interval]] = {}
317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319     pinned_model = (LogisticRegression.load(variant.classifier_model)
320                     if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                  for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),

```

```

351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
361         mean)."""
362         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
363         _METRICS}
364
365     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
366     Dict[str, Any]:
367         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
368         hides)."""
369         gts_by_name = {v.name: (v.gts or []) for v in videos}
370         overlaps = _segm.DEFAULT_OVERLAPS
371         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
372         ratios: List[float] = []
373         for name, preds in eval_v.get("predictions", {}).items():
374             gts = gts_by_name.get(name, [])
375             got = _segm.f1_at_overlaps(preds, gts, overlaps)
376             for ov in overlaps:
377                 f1k[ov].append(got[ov])
378             ratios.append(_segm.segment_count_ratio(preds, gts))
379
380     def _mean(xs: List[float]) -> float:
381         return sum(xs) / len(xs) if xs else 0.0
382
383     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
384     f1k.items()}
385     out["segment_count_ratio"] = _mean(ratios)
386     return out
387
388     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
389     eval_b: Dict[str, Any]) -> Dict[str, Any]:
390         """C1 + C4 honest reporting layered over a compare() pair (additive,
391         EXPERIMENT_HARNESS §6).
392
393         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
394         order),
395         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
396         when
397         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
398         """
399         a_f1 = [p["f1"] for p in eval_a["per_video"]]
400         b_f1 = [p["f1"] for p in eval_b["per_video"]]
401         return {
402             "variant_a_ci": _ci_block(eval_a),
403             "variant_b_ci": _ci_block(eval_b),
404             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
405             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
406                             "variant_b": _segmentation_block(videos, eval_b)},
407         }
408
409     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
410     iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
411     Dict[str, Any]:
412         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
413         same

```

```

406         golden corpus and reports the B ? A deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
409         `calibration.pct_improvement`;
410         learned variants are scored LOVO-honestly. The result is a self-contained record fit
411         for
412         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
413         it.
414         ``honest_stats`` (default True) adds a ``"honest"`` block ? per-metric bootstrap
415         CIs, a
416         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
417         *alongside* the
418         existing bare-mean fields (additive: nothing existing changes). Set False for the
419         legacy
420         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
421         §13/C1+C4): a
422         bare LOVO mean at n?6 is not trustworthy on its own.
423         """
424         if len(videos) < 1:
425             raise ExperimentError("compare needs at least one labeled video")
426         eval_a = evaluate_variant(videos, variant_a, iou, honest)
427         eval_b = evaluate_variant(videos, variant_b, iou, honest)
428
429         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
430         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
431         m in _METRICS}
432
433         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
434         per_video_delta = [
435             {"video": p["video"], {m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
436             for p in eval_b["per_video"]]
437         ]
438
439         # One fingerprint over the whole label set ? detects "the deltas were measured
440         against
441         # different labels" the same way regression.gt_hash guards the no-regression
442         baseline.
443         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
444
445         result: Dict[str, Any] = {
446             "iou": iou,
447             "label_set_hash": gt_hash(all_gts),
448             "num_videos": len(videos),
449             "variant_a": eval_a,
450             "variant_b": eval_b,
451             "delta": delta,
452             "delta_pct": delta_pct,
453             "per_video_delta": per_video_delta,
454         }
455         if honest_stats:
456             result["honest"] = honest_summary(videos, eval_a, eval_b)
457         return result
458
459     # ----- SxS on unlabeled video
460
461     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
462         agree_iou: float = 0.5) -> Dict[str, Any]:
463         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
464
465         With no labels there is no lift to measure ? instead this surfaces where the two
466         variants agree vs diverge, which is exactly what a human reviews (and what two
467         highlight reels would show side by side):

```

```

461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465       the human / a later golden label adjudicates).
466
467 Both variants must be predictable without labels: a learned variant therefore needs
a
468 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469 ``metrics.match_intervals`` ? no new matching logic.
470 """
471 preds_a = predict(variant_a, video)
472 preds_b = predict(variant_b, video)
473 mr = match_intervals(preds_a, preds_b, agree_iou)
474
475 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476           for m in mr.matches]
477 agree_ious = [m.iou for m in mr.matches]
478 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481 def _dur(ivs: List[Interval]) -> float:
482     return sum(e - s for s, e in ivs)
483
484 return {
485     "video": video.name,
486     "agree_iou": agree_iou,
487     "variant_a": variant_a.name,
488     "variant_b": variant_b.name,
489     "num_a": len(preds_a),
490     "num_b": len(preds_b),
491     "num_agreed": len(agreed),
492     "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493     "duration_a": _dur(preds_a),
494     "duration_b": _dur(preds_b),
495     "agreed": agreed,
496     "a_only": a_only,
497     "b_only": b_only,
498 }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a ``compare()`` result to the JSON
507     leaderboard (``EXPERIMENT_HARNESS.md`` §6: experiments are records). Generalises
508     ``regression_baseline.json``; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries

```

```

the
525     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                 row["variant_a"]["name"], row["variant_b"]["name"],
539                 (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558         rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563         rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566         rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568         gts=gts)
569
570

```

6. assistant (2026-06-16T02:32:12.536Z)

Now let me read the key files for the seam choices: fusion_compare, distill_local, windowing, regression, and the existing per-video regression test template.

7. user (2026-06-16T02:32:13.016Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
2     set.
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
4     `fusion_golden.py`
5     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
6     out) ?
7     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
8     harness's
9     designed seam): the person features are extra columns the learned LogisticRegression

```

```

weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10    python -m backend.eval.fusion_compare --features-dir output --record
11    """
12    from __future__ import annotations
13
14    import argparse
15    import glob
16    import json
17    import os
18    from typing import Any, Dict, List
19
20    from backend.eval import experiment
21    from backend.eval.eval_stats import paired_delta_ci
22    from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25    def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26        """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27        (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28        by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29        by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30        by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31        vids = [p["video"] for p in res["variant_a"]["per_video"]]
32        return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35    def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36        """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37        feature columns + golden gts). The harness derives labels from gts itself."""
38        videos: List[experiment.VideoCandidates] = []
39        for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40            with open(path, encoding="utf-8") as f:
41                d = json.load(f)
42                cands = d["candidates"]
43                intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                features: Dict[str, List[float]] = {}
45                for fn in SHUTTLE_FEATURE_NAMES:
46                    features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                for fn in PERSON_FEATURE_NAMES:
48                    features[fn] = [float(c["person"][fn]) for c in cands]
49                gts = [(float(a), float(b)) for a, b in d["gts"]]
50                videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                            features=features, gts=gts))
52        return videos
53
54
55    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                threshold=threshold)
59        fusion = experiment.Variant(name="shuttle_person_fusion",
60                feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                threshold=threshold)
62        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65    def format_result(res: Dict[str, Any], iou: float) -> str:

```

```

66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

8. user (2026-06-16T02:32:15.964Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10    baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.

```

```

16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78

```



```

79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                       f1: float, path: str = DEFAULT_BASELINE_PATH,
82                       iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                       gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100                    _baseline_key(video_id, stage), precision, recall)
101
102     # ----- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105               stage: str = "final", iou_threshold: float = 0.5,
106               detector: Optional[str] = None,
107               tolerance: float = DEFAULT_TOLERANCE,
108               baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
110         baseline.
111
112         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
113         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
114         precision OR recall falls more than `tolerance` below the snapshotted baseline.
115         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
116         caller can choose to snapshot the current numbers via `save_baseline`.
117         """
118         preds = get_predictions(db, video_id, stage, detector=detector)
119         s = metrics.score(preds, ground_truth, iou_threshold)
120         fp = gt_hash(ground_truth)
121
122         baselines = load_baselines(baseline_path)
123         b = baselines.get(_baseline_key(video_id, stage))
124
125         reasons: List[str] = []
126         regressed = False
127         has_baseline = b is not None
128         base_p = base_r = None
129         if b is not None:
130             base_p, base_r = b["precision"], b["recall"]
131             # Incommensurable comparison guard: a baseline taken at a different IoU/detector
132             or
133             # against different labels is not a valid reference ? flag it instead of
134             trusting it.
135             b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
136             b.get("gt_hash")
137             if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
138                 regressed = True
139             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
140             (incommensurable)")
141             if b_det is not None and b_det != detector:

```

```

137         regressed = True
138         reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )
158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                          tier: str = "file", stage: str = "final",
162                          iou_threshold: float = 0.5,
163                          detector: Optional[str] = None,
164                          tolerance: float = DEFAULT_TOLERANCE,
165                          baseline_path: str = DEFAULT_BASELINE_PATH,
166                          guideline: Optional[str] = None,
167                          sport: str = "badminton",
168                          provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172     once those providers land). `video_ref` is whatever that provider expects (a CSV
173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                  detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182 def format_result(r: RegressionResult) -> str:
183     verdict = "REGRESSED" if r.regressed else "ok"
184     head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185            f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186     if r.baseline_precision is not None:
187         head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188     return head + "\n    - {why}" for why in r.reasons)
189

```

9. user (2026-06-16T02:32:18.911Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2

```

```

3     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4     ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5     MagicMock runner). The offline ablation litmus
6     (`test_ablation.test_litmus_reproduces_gate_a`)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9     runs the real ``FusionSegmenter`` decision seams (`enrich_candidate_stats` net-
crossings +
10    ``_select_for_handoff`` Gate A) over windows from the **production**
11    ``trajectory_to_action_windows`` config, on the 6 cached golden WASB trajectories, and
asserts
12    the served path's measured behaviour.
13    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14    CI where the golden trajectories are absent; runs on the owner box where they are
present.
15
16    HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing**
Gate A is
17    NOT the +0.074 win the offline harness reports. Production windowing is already
conservative
18    (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg
ratio
19    already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served
?F1 is
20    ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the
permissive
21    proposal windowing (asserted in `test_proposal_windowing_reproduces_offline_lift`
below ? that
22    is what the offline litmus measures). These tests pin BOTH facts so a future windowing
change
23    that silently revives ? or breaks ? either one trips the litmus.
24    ""
25    from __future__ import annotations
26
27    import json
28    import os
29
30    import pytest
31
32    # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
33    # (CI), skip the whole module rather than erroring at import.
34    pytest.importorskip("cv2")
35
36    from backend.eval import served_gate_a as sga # noqa: E402
37    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
38    from backend.pipeline.detectors import rally_gate # noqa: E402
39    from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
40    from backend.eval.metrics import score # noqa: E402
41    from backend.eval import segmentation_metrics as _segm # noqa: E402
42
43    # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
44    # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
45    # out-of-tree checkout.
46    _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
47    MANIFEST = os.environ.get(
48        "SERVED_GATE_A_MANIFEST",
49        os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
50    )

```

```

51
52
53 def _golden_specs():
54     """Return the manifest specs whose trajectory + label files are all present, else
55     []."""
56     if not os.path.isfile(MANIFEST):
57         return []
58     with open(MANIFEST, encoding="utf-8") as f:
59         specs = json.load(f)
60     ok = [s for s in specs
61           if os.path.isfile(str(s.get("trajectory", "")))
62           and os.path.isfile(str(s.get("labels", "")))]
63     return ok if len(ok) >= 6 else []
64
65 _SPECS = _golden_specs()
66 _skip = pytest.mark.skipif(
67     not _SPECS,
68     reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
69 )
70
71
72 # ----- served-path
litmus
73
74 @_skip
75 def test_served_path_runs_on_all_six_golden_videos():
76     """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
77     all 6
78     golden trajectories and yields a handoff window list for each, for both gate
79     settings."""
80     results = sga.evaluate_manifest(_SPECS)
81     assert len(results) == 6
82     for r in results:
83         assert r.num_gt > 0
84         assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows,
85         never adds
86
87 @_skip
88 def test_served_gate_a_does_not_regress_f1():
89     """SERVED no-regression guard: on the production windowing Gate A must not
90     MATERIALLY hurt
91     F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already
92     merged
93     away by the conservative production windowing) ? so we assert the mean ?F1 sits in a
94     tight
95     band around 0, never a real regression. (The +0.074 lift lives under the permissive
96     proposal
97     windowing ? see the dedicated test below.)"""
98     results = sga.evaluate_manifest(_SPECS)
99     agg = sga.aggregate(results)
100     # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band
101     so a
102     # real regression (Gate A actively destroying served F1) trips, while the measured
103     ~neutral
104     # result passes.
105     assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
106     # And Gate A must never collapse served F1 to ~0 on the production path.
107     assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
108
109 @_skip
110 def test_served_gate_a_does_not_increase_over_segmentation():
111     """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when

```

```

it's on
105     (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
106     results = sga.evaluate_manifest(_SPECS)
107     agg = sga.aggregate(results)
108     assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
109     # Per-video too: the gate is monotone (kept windows ? all windows).
110     for r in results:
111         assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
112
113
114     @_skip
115     def test_served_uses_production_windowing_operating_point():
116         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
117         production windowing, this litmus's premise (and the divergence it documents) is
revisited
118         deliberately, not silently."""
119         assert sga.PROD_VELOCITY_THRESH == 0.02
120         assert sga.PROD_MERGE_GAP == 2.0
121         assert sga.PROD_MIN_WINDOW == 2.0
122
123
124     # ----- the +0.074 lives under PROPOSAL
windowing
125
126     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
127     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
128     # small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served
one.
129     _PROPOSAL = (0.005, 1.0, 0.5)
130
131
132     @_skip
133     def test_proposal_windowing_reproduces_offline_lift():
134         """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal
windowing, the
135         SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline
litmus
136         (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result
is a
137         WINDOWING difference, not a broken gate ? and locks the offline number to its true
premise."""
138         d_on, d_off, seg_on, seg_off = [], [], [], []
139         for s in _SPECS:
140             fps, fw = float(s["fps"]), float(s["frame_width"])
141             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
142             wins = TrackNetRunner.trajectory_to_action_windows(
143                 pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
144                 merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
145             iv = [(w["start"], w["end"]) for w in wins]
146             est = rally_gate.estimate_play_axis(pts)
147             gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
148             on = [(g.start, g.end) for g in gated if g.crossings >= 2]
149             gts = load_golden_gts(s["labels"])
150             d_on.append(score(on, gts).f1)
151             d_off.append(score(iv, gts).f1)
152             seg_on.append(_segm.segment_count_ratio(on, gts))
153             seg_off.append(_segm.segment_count_ratio(iv, gts))
154         n = len(d_on)
155         mean_delta = (sum(d_on) - sum(d_off)) / n
156         wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
157         assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
158         assert wins == 6, wins

```

```
159         assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
160         assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)
161
```

10. assistant (2026-06-16T02:32:22.522Z)

This is highly load-bearing. Now let me look at the windowing module, distill_local, fusion_golden's schema constants, and the GOLDEN_DATA_SHARING doc that's the policy conflict.

11. user (2026-06-16T02:32:23.465Z)

```
1      """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3      Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4      further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5      using a few resolution/fps-INVARIANT local features, so the model transfers across
6      videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8      Flow (all local at runtime):
9          WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10             filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11             active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12             -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37      from backend.eval import windowing as _windowing
38
39      Interval = Tuple[float, float]
40
41      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42      # Recall-oriented proposal: deliberately permissive so real rallies are among the
43      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44      # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45      # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46      # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47      # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48      PROPOSAL = _windowing.PROPOSAL.as_tuple()
49      BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
```

```

50
51
52     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
53                          proposal: Tuple[float, float, float] = PROPOSAL) ->
54     Tuple[List[Interval], List[List[float]]]:
55         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
56         rows)."""
57         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
58         vt, mg, mwd = proposal
59         wins = TrackNetRunner.trajectory_to_action_windows(
60             points, fps=fps, frame_width=frame_width,
61             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
62         intervals = [(w["start"], w["end"]) for w in wins]
63         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
64         crossings
65         X: List[List[float]] = []
66         for w, g in zip(wins, gated):
67             dur = max(1e-6, w["end"] - w["start"])
68             win_frames = max(1.0, dur * fps)
69             X.append([
70                 dur, # rally length (sec) ?
71                 float(w["peak_velocity"]), # already normalized by
72                 float(w["mean_velocity"]),
73                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
74                 float(g.crossings), # net crossings (count) ?
75                 the rally signal
76             ])
77         return intervals, X
78
79
80     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
81     List[Interval]:
82         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
83         vt, mg, mwd = BASELINE_LOCAL
84         wins = TrackNetRunner.trajectory_to_action_windows(
85             points, fps=fps, frame_width=frame_width,

```

12. user (2026-06-16T02:32:26.392Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4     ?
5     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6     built
7     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9     pipeline
10    (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11    merge_gap=2.0,
12    min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13    is
14    measured on candidates **production never serves**, so a "win" need not transfer. This
15    is
16    exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
17    set
18    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
21    the
22    SERVED preset is **single-sourced from the same Pydantic config defaults production
23    reads**

```

```

15     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the **what windowing** ; that one is the **promotion
gate**.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61
62         def as_tuple(self) -> Tuple[float, float, float]:
63             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
64             ``trajectory_to_action_windows`` / ``distill_local`` call sites already
speak."""
65             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
66
67         def to_dict(self) -> Dict[str, Any]:

```



```

68         return {
69             "name": self.name,
70             "velocity_thresh": self.velocity_thresh,
71             "merge_gap": self.merge_gap,
72             "min_window_duration": self.min_window_duration,
73         }
74
75
76     def _served_preset_from_defaults() -> WindowingPreset:
77         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
78         reads.
79         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
80         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
81         wasb/fusion/tracknet)
82         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
83         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
84
85         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
86         the whole
87         point: change a served default in ``config/models.py`` and the eval SERVED preset
88         follows,
89         so the two can never silently diverge again.
90         """
91         idx = IndexingConfig() # construct with defaults ? the served operating point
92         return WindowingPreset(
93             name="served",
94             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
95             default 0.02
96             merge_gap=float(idx.dead_time_merge_gap),
97             min_window_duration=float(idx.min_action_window_duration),
98         )
99
100     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
101     cue is
102     #: only promotable if it holds up HERE ? the operating point real users get.
103     SERVED: WindowingPreset = _served_preset_from_defaults()
104
105     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
106     (deliberately
107     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
108     ? a
109     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
110     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
111     valid.
112     PROPOSAL: WindowingPreset = WindowingPreset(
113         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
114     )
115
116     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
117     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
118
119
120     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
121         """The SERVED windowing a *specific* live config serves (overrides honoured).
122
123         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
124         an
125         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
126         ``family``
127         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
128         served
129         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
130         operating point;

```

```

120         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
121         """
122         idx = _as_mapping(config, "indexing")
123         family_cfg = _as_mapping(idx, family)
124         return WindowingPreset(
125             name=f"served:{family}",
126             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
127             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
128             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
129         )
130
131
132     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
133         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
134         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
135         if obj is None:
136             return {}
137         if isinstance(obj, Mapping):
138             sub = obj.get(key, {})
139         elif hasattr(obj, "get"):
140             sub = obj.get(key, {})
141         else:
142             sub = getattr(obj, key, {})
143         return sub if isinstance(sub, Mapping) else {}
144
145
146     def build_windows(points: List[Any], fps: float, frame_width: float,
147                       preset: WindowingPreset) -> List[Dict[str, Any]]:
148         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
149
150         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
151         instead of three loose floats, so eval candidate-building and the served path
reference the
152         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
153         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
154
155         vt, mg, mwd = preset.as_tuple()
156         return TrackNetRunner.trajectory_to_action_windows(
157             points, fps=fps, frame_width=frame_width,
158             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
159
160
161     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
162         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
163         return [(w["start"], w["end"]) for w in windows]
164

```

13. assistant (2026-06-16T02:32:29.139Z)

Now let me check the fusion_golden schema constants, the GOLDEN_DATA_SHARING policy doc, and the EVALUATION doc to know where to document.

14. user (2026-06-16T02:32:30.219Z)

```

1         """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2

```

```

3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
44         labels = str(spec["labels"])
45         if labels.endswith(".rallies.csv"):
46             return labels[: -len(".rallies.csv")] + ".MP4"
47         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                       frame_skip: int = 3, iou: float = 0.5,
52                       log_every_sec: float = 15.0,
53                       telemetry: Optional[Any] = None) -> Dict[str, Any]:
54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns `{name, fps, frame_width,

```

```

candidates:[{start,end,shuttle{},person{},label}], gts}``.
57     `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
58     when None a real `PersonDetector` is built (loads torchvision on first use).
59     `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60     from the detection progress callback so a hard kill leaves a forensic runlog.
61     """"
62     fps, fw = float(spec["fps"]), float(spec["frame_width"])
63     traj = str(spec["trajectory"])
64     points = TrackNetRunner.parse_trajectory_csv(traj)
65     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66     gts = load_golden_gts(str(spec["labels"]))
67     labels = label_candidates(intervals, gts, iou)
68     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70     axis, net_px, _span = rally_gate.estimate_play_axis(points)

```

15. user (2026-06-16T02:32:32.769Z)

```

1     # Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions
2
3     **Status:** convention + plan (2026-06-15). Tracked in **#175**.
4     **Purpose:** a single cloud-synced location for the GPU-extracted golden artifacts
(trajjectory CSVs, `_golden_features.json`, manifests, baseline snapshots) so the **GPU box**
and **CPU dev/agent sessions** share the same files ? without committing large data to git.
5
6     ## Why this exists
7     The rally-quality measurement is split across two machines by capability:
8     - **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the
**videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL**
stack ? only the GPU box can run it.
9     - **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO
**litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the
small `_golden_features.json` ? any session (incl. a CPU dev container) can run them.
10
11     Today those artifacts sit in gitignored `output/` (per-machine) and scattered local
folders (`...\Annotation Setup\Collect\Trajectories`, `...\Golden Labelled`). Nothing syncs
them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap
with OneDrive.
12
13     ## Canonical shared folder
14     `%USERPROFILE%\OneDrive\rally-annotator\golden-data\` ?
`C:\Users\avidu\OneDrive\rally-annotator\golden-data\`
15
16     OneDrive is present + signed-in on both the GPU box and the dev machine under the same
account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an
overwrite):
17
18     ```
19     golden-data/
20         README.md
21         features/<corpus-tag>/      *_golden_features.json    (small ? the key shared
artifact)
22         trajectories/<corpus-tag>/  *_predict.csv / *_wasb.csv (medium ? reusable across
runs)
23         manifests/                  *_lovo_manifest.json
24         baselines/                  leaderboard / regression snapshots (or keep tracked in
eval_baselines/)
25     ```
26
27     `<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos
here (tens of GB) ? keep those local to the GPU box.

```

```

28
29     ## The GPU ? CPU workflow
30     1. **GPU box** (videos + CSVs + labels present):
31         ```
32         python -m backend.eval.fusion_golden --manifest <manifest> \
33             --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
data\features\2026-06-15-n6"
34         ```
35         (or extract to local `output/` then copy the `_golden_features.json` into the shared
folder).
36     2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
37     3. **CPU dev/agent session** points the analysis at the shared folder:
38         ```
39         python -m backend.eval.fusion_compare --features-dir "...golden-
data\features\2026-06-15-n6" --record
40         python -m backend.eval.ablation --features-dir "...golden-
data\features\2026-06-15-n6" --granularity group
41         ```
42         ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/`
to also light up the `skipif` golden-litmus tests
(`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).
43
44     ## Optional code ergonomics (in #175, not required)
45     - A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` /
`fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` /
`--features-dir` flags already make this a convention; the env var is just ergonomics.)
46     - A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared
folder.
47
48     ## Guardrails
49     - The shared folder lives **outside the repo** ? never committed. Keep `output/`
gitignored and do **not** symlink the shared folder into a tracked path.
50     - The features derive from the **Proprietary** corpus ? keep the OneDrive folder
**private** (no public share links).
51
52     See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor),
`docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector
setup).
53

```

16. assistant (2026-06-16T02:32:35.975Z)

Now let me check the EVALUATION.md and EXPERIMENT_HARNESS.md docs to decide the doc home, and look at the metrics module and gt_loader quickly to confirm determinism contracts.

17. user (2026-06-16T02:32:36.165Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is

```



```

82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104 def score(preds: List[Interval], gts: List[Interval],
105          iou_threshold: float = 0.5,
106          match_result: Optional[MatchResult] = None) -> Score:
107     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110     extracting the failure lists); otherwise matching is done here.
111     """
112     mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114
115     tp = len(mr.matches)
116     fp = len(mr.unmatched_pred_indices)
117     fn = len(mr.unmatched_gt_indices)
118
119     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122 else 0.0
123
124     if mr.matches:
125         mean_iou = sum(m.iou for m in mr.matches) / tp
126         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127 mr.matches) / tp
128         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129 mr.matches) / tp
130     else:
131         mean_iou = mean_start_err = mean_end_err = 0.0
132
133     return Score(
134         iou_threshold=iou_threshold,
135         num_pred=len(preds), num_gt=len(gts),
136         true_positives=tp, false_positives=fp, false_negatives=fn,
137         precision=precision, recall=recall, f1=f1,
138         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139     )
140
141
142 def pr_curve(preds: List[Interval], gts: List[Interval],
143             thresholds: Optional[List[float]] = None) -> List[Score]:
144     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
145
146     Lets you see strict-vs-lenient behaviour rather than a single operating point.

```

```

143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

18. assistant (2026-06-16T02:32:40.137Z)

Now let me check the eval_stats for determinism (seed) and look at the EVALUATION.md doc head to decide doc home, plus the existing eval_baselines file shapes.

19. user (2026-06-16T02:32:42.134Z)

Annotation & Evaluation

How to measure how well the pipeline detects rallies, using hand-annotated ground-truth timestamps. The harness scores predictions **already stored in the SQLite DB** ? it makes no API calls and is fully deterministic, so you can run it as often as you like for free.

1. What you annotate (and what it does / doesn't do)

You annotate, per video, the `[start, end]` time range of each **rally**. That's all the evaluation harness needs.

```

> Important: these rally timestamps measure and tune rally segmentation
> ? i.e. where the pipeline says rallies begin and end. They do not train
> the shuttle detector (TrackNet/WASB), which learns from per-frame shuttle
> `(x, y)` coordinate labels, a different and much heavier annotation. Use rally
> timestamps as your golden evaluation set; only invest in frame-level
> coordinate labels if evaluation shows detection (not segmentation) is the
> bottleneck. See `docs/TRACKNET_WSL_SETUP.md`.

```

2. Annotation file format (CSV)

One CSV per video, one rally per row:

```

```csv
start,end
00:01:12,00:01:39
102.5,131.0
```

```

- The `start,end` header is optional (auto-detected).
- Timestamps may be **decimal seconds** (`102.5`) or **colon-separated** `MM:SS` / `HH:MM:SS` (`00:01:12`). Forms can be mixed.
- Blank lines and `#`` comments are ignored.
- Malformed rows (bad timestamp, `start >= end`) fail loudly rather than being silently skipped ? so labeling mistakes surface immediately.

Generate an empty template to fill in:

```
=== EXPERIMENT_HARNESS head ===
```

Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by

```

Status: P1 IMPLEMENTED (`backend/eval/experiment.py` + `tests/test_experiment.py`,
shipped
2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest), `compare_unlabeled()`
(SxS on NEW unlabeled footage), and the leaderboard. P2?P4 remain owner-gated. Phases below
are
sequenced but each is a separate PR.

```



```

> **Companion docs:** the cues this experiments *on* are in
> [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
> [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels + `baseline.json` come
from
> [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is
**ADR-007**
> ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled classifier
is
> **ADR-004**. The 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).

---

```

1. Context ? why this, why now

PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues, all
**default
OFF**. The owner's directive for *how* we evolve quality from here:

```

> "Evolve the codebase so we can keep finding new ways to improve quality and experiment with
them ?
> or add them as a signal ? without suffering any regressions if the idea doesn't work out.
Rolling
> back the code would be too dangerous."

```

```

=== eval_stats seed ===
18:deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
33:         n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34:     """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
45:     rng = random.Random(seed)
58:         n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
64:     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
91:         confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
101:     summary = mean_with_ci(deltas, confidence, n_boot, seed)

```

20. assistant (2026-06-16T02:32:45.919Z)

I now have enough grounding. Let me quickly confirm the CODE_MAP \$0 file-placement table mention
and the gt_loader hash helper, then synthesize.

21. user (2026-06-16T02:32:46.708Z)

```

7:? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git
history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-
cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
`segment_watermarks`/\`prune_segments`); detector artifacts are keyed by **video_id**, not
basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
`match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked
`eval_baselines`/\` and **exits 3 on a missing baseline**; the API validates
`video_path`/\`output_filename`. New tests: `test_api_endpoints`, `test_compiler`,
`test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`,
`test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`,
`test_import_guard`, `test_cvat_robustness`).
32:| A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
36:| Eval baseline / leaderboard JSON | `eval_baselines`/\` | Tracked (survives a clean checkout);
the no-regression gate reads it. |
42:(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py`
moved to
141:- `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ? SHARED
ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix BOTH"
footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider
handoff -> P4 DB/link; single `::shot_count_from` (old wasb/tracknet shot_count drift gone).

```

```

NOT registered itself. ? Subclass contract: `family` (config section
`indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir,
`<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description`
properties, `_build_runner(idx_cfg) -> (DetectorRunner, detection_params extras)`. **Two fusion
seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test):
`:::enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4
motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate)
**and `:::select_for_handoff(windows>window_stats,idx_cfg)`** (base = all indices). ? shared
knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass;
needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by
`tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the
base-hook-identity tests ? parametrized over both twins).
142:- `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name `fusion_hybrid`,
**default-OFF** multi-signal fusion P2) ? subclasses the engine; `family='fusion'`,
`display_label='Fusion'`. `:::build_runner` selects the substrate via
`indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config.
`:::enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via
`rally_gate.gate_windows(..., estimate=self.axis_estimate(points))` ? axis/net cached per
trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features
(`:::person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-
video frame range ? `fusion_features.compute_person_features`). `:::select_for_handoff`
overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`,
person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full
superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a
tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads
`idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3
cleanup). Tests: `tests/test_fusion_hybrid.py`.
158:- `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue feature
math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame
`{frame_idx:[(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon`
+ `net=(axis,pos)`. `:::point_in_polygon` (ray-cast), `:::person_in_court` (foot-point; None
polygon=count-all; bad dims=False), `:::in_court_counts`, `:::players_in_court` (median),
`:::in_court_ratio`, `:::mean_player_motion` (per-track centre displacement; x by width, y by
height), `:::two_sided_motion` (both net-halves occupied), `:::shuttle_player_colocation` (shuttle
inside a person box  $\pm$  pad_frac halo over aligned visible frames ? **the held-shuttle
discriminator**), `:::compute_person_features` (aggregator ? the 5 `:::PERSON_FEATURE_NAMES`). All
scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
178:- `@backend/eval/regression.py` ? CI gate. `:::DEFAULT_TOLERANCE=0.05`,
`:::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored
output/), `:::regress`, `:::regress_with_provider` (GT via ? `annotation_registry.get(tier)`;
"file"->`FileProvider`/"vendor"->`HumanVendor`/"auto"->`oracle`,
`:::save_baseline`/`:::load_baselines`, `:::gt_hash` (label-set fingerprint, reused by
experiment.py), `:::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI
`run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
179:- `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin
"experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes
metrics/calibration/classifier/training/regression ? NO new scoring math).
`:::Variant` (segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate
B],classifier_model,feature_names,threshold,merge_gap;
`to_dict`/`from_dict`/`spec_hash`/`is_learned`), `:::CONTROL` (pinned zero-gate baseline).
`:::VideoCandidates` (name,intervals,features{col-major},gts), `:::predict` (gate?learned-
filter?`gemini_refine.merge_overlaps`), `:::evaluate_variant` (static / pinned-model / **honest-
LOVO** train-on-others ? mirrors distill_local), `:::compare` (labeled **B?A** deltas +
`label_set_hash`), `:::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via
`match_intervals`, no GT ? the owner's "SxS on new videos"), `:::record_experiment` (?
`eval_baselines/experiment_leaderboard.json`), `:::load_video_candidates` (DB bridge via
`query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails
gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg
invariant (tested): all cues OFF ? byte-identical to CONTROL.**
216:- `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the
footgun RULES live in spec.yaml, not here). `:::record_run(...)` ? top-level assembler (does NOT
write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`,
runs 4 stage builders + `_highlights`, `rec.finalize()``.

```

```

`::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters
here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now
also `audio_present` for Input Quality reports extension; $ `fps_source` ?
{unknown,probed,fallback}), `::validation_stage`, `::segmentation_stage` ($ `details.{segment
er_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_request
ed,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"),
`::ai_stage` ($ `details.{ai_based,provider,model,api_key_present,oversize_clips_skipped}`;
metric `confirmed_segments`), `::highlights` (coverage_pct). `record_run(run_signals=)` carries
`run_signals.pop(video_id)` counters into `_ai_stage`. ? `record_run` fps fallback:
`fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
`::SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so
pure helpers like video_meta are importable for tests/coverage even without the dep.)
327:| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
336:## 8. TEST MAP (`tests/test_*.py` -> subsystem)

```

22. assistant (2026-06-16T02:32:51.181Z)

I now have all the grounding I need. I've confirmed the real modules, the determinism contracts (seeded bootstrap, greedy deterministic matcher, the silent fps/frame_width fallback hazard), the policy conflict in GOLDEN_DATA_SHARING.md, the windowing preset single-sourcing that closes the eval?serve trap, and the existing per-video regression test template. Let me synthesize the unified design.

23. user (2026-06-16T02:35:48.150Z)

Structured output provided successfully

24. assistant (2026-06-16T02:36:05.083Z)

I've delivered the unified design via StructuredOutput. Here is a brief summary of the key decisions and how I resolved the conflicts between the three proposals.

****Core synthesis:**** Freeze at the trajectory CSV seam (Proposal 3's choice, the true GPU?pure-Python handoff and widest deterministic surface) but build the full dual-gate + version/provenance machinery and synthetic-first/two-tier licensing reconciliation (Proposals 2/3). Fixtures live in a new `tests/fixtures/golden\_regression/` (test inputs belong under tests/), while the quality-floor baseline fills the reserved-but-absent `eval\_baselines/regression\_baseline.json` (no-regression score artifact belongs in eval_baselines/) ? honoring both CODE_MAP \$0 conventions rather than overloading eval_baselines/ with raw fixtures as Proposal 1 did.

****Conflicts resolved explicitly:****

- ***Which seam:** trajectory CSV over feature-JSON-only ? the latter is blind to the windowing brain (`trajectory\_to\_action\_windows`) and Gate-A `net\_crossings`, the code most likely refactored. Confirmed against `backend/eval/distill\_local.py::build\_candidates` and `windowing.py`.
- ***One gate vs two:** both ? a characterization/snapshot gate (catches refactor side-effects at constant F1) AND a quality-floor gate vs golden labels (catches real quality drops, tied to the ablation track).
- ***Privacy policy conflict:** the design reverses `docs/GOLDEN\_DATA\_SHARING.md`'s "never commit proprietary-derived artifacts" posture, so it ships synthetic tier-0 with NO owner gate and defers real tier-1 to an owner-approved PR.

****Load-bearing grounding I verified in code:****

- `metrics.match\_intervals` is greedy-deterministic (sort key `(-iou,pi,gi)`), empty?0.0 not NaN ? stable asserts.
- `eval\_stats` bootstrap/sign-test is seeded (`seed=0`) ? snapshot-able honest stats.
- The silent `fps<=0?30.0 / frame\_width<=0?1280.0` fallback in windowing ? fixtures MUST pin (fps, frame_width) in provenance.
- `windowing.SERVED` is single-sourced from `IndexingConfig` defaults ? the fix for the eval?serve Gate-A trap (`test\_served\_gate\_a\_regression.py` pins +0.074 proposal vs ?0.008 served), so the snapshot records its windowing preset.
- `regression.py`'s `gt\_hash`/incommensurability guards and `experiment.py`'s `label\_set\_hash`/`spec\_hash` are reused for keying ? no parallel scheme invented.

Six open owner questions are flagged (chiefly: approve tier-1 real-fixture commit; repo public/private; which n=6 videos are minors-free/license-clean). Honest limits are stated plainly ? chiefly that characterization locks behaviour (including current bugs), not correctness, and does not cover the detector/decode/provider/stitch; and that the dev container here cannot run the full pipeline, so tier-1 extraction and the owner-box DB regression were not verifiable in this environment.